



## **FİNAL PROJE RAPORU**

**Ders: BİL-204 \ Yazılım Mühendisliği**

**Bölüm/Şube: Bilgisayar Mühendisliği**

**Grup Adı: Grup - 4**

**Proje Adı:Proof-Based-Portfolio Platform**

**Teslim Tarihi: 24.05.2026**

### **Grup Üyeleri:**

**- Ali Akbayrak (25100011409)**

**- Elif İpek Kaynar (24100011007)**

**- Mustafa Pekkirışci (25100011405)**

**-Rabia Şahin (25100011404)**

**-Selime Buse Yanal (24100011817)**

**-Şeyda Öksüzoğlu (25100011471)**

# İçindekiler

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>1. GİRİŞ</b>                                               | <b>4</b>  |
| 1.1. Raporun Amacı ve Bağlamı                                 | 4         |
| 1.2. Sistemin Amacı ve Çözüm Sunduğu Temel Problemler         | 4         |
| 1.3. Projenin Nihai Kapsamı ve Modüler Sınırları              | 4         |
| <b>2. Proje Deneyimi</b>                                      | <b>5</b>  |
| <b>3. Yapay Zeka Kullanımı</b>                                | <b>8</b>  |
| 3.1. Kullanılan Yapay Zeka Araçları ve Kapsamı                | 8         |
| 3.2. Sürüm Kontrolü ve Eş-Geliştirme (Co-Authoring) Kanıtları | 8         |
| 3.3. Yapay Zekanın Katkısı ve Mühendislik Denetimi            | 9         |
| <b>4. Kullanıcı Kılavuzu</b>                                  | <b>10</b> |
| 4.1. Ana Sayfa (Landing Page) — /                             | 10        |
| 4.2. Kayıt Ol (Register) — /register                          | 12        |
| 4.3. Giriş Yap (Login) — /login                               | 13        |
| 4.4. Parola Sıfırlama — /reset-password                       | 14        |
| 4.5. Dashboard — Portföy Oluşturma (Onboarding)               | 16        |
| 4.6. Dashboard — Ana Sayfa (/dashboard)                       | 19        |
| 4.7. Dashboard — Profil Düzenleme                             | 21        |
| 4.8. Dashboard — Proje Yönetimi                               | 21        |
| 4.9. Dashboard — Bağlantı Yönetimi                            | 25        |
| 4.10. Dashboard — GitHub Import                               | 27        |
| 4.11. Dashboard — Yayınlama ve Tema                           | 29        |
| 4.12. Public Portföy Sayfası — /[slug]                        | 31        |
| 4.13. Ayarlar — Hesap Silme (/settings)                       | 33        |
| 4.14. Hata Sayfaları ve Diğer                                 | 35        |
| 4.15. Kullanıcı Roller ve Erişim Matrisi                      | 37        |
| <b>5. Kurulum Talimatları</b>                                 | <b>38</b> |
| 5.1. Gereksinimler                                            | 38        |
| 5.2. Repository'yi Klonlama                                   | 38        |
| 5.3. Bağımlılıkları Yükleme                                   | 39        |
| 5.4. Ortam Değişkenlerini Ayarlama                            | 39        |
| 5.5. Veritabanı Migration                                     | 40        |
| 5.6. Seed Verisi (Opsiyonel)                                  | 41        |
| 5.7. Geliştirme Sunucusunu Başlatma                           | 41        |
| 5.8. Diğer Yardımcı Komutlar                                  | 41        |
| 5.9. Testleri Çalıştırma                                      | 42        |
| 5.10. Production Build                                        | 43        |
| 5.11. Vercel Deploy                                           | 43        |
| 5.12. Proje Klasör Yapısı                                     | 43        |
| <b>6. Görev Dağılımı</b>                                      | <b>45</b> |

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| <b>7. Yaptıklarımız, Yapmadıklarımız ve Nedenleri.....</b>             | <b>47</b> |
| 7.1. Gerçekleştirilen ve Gerçekleştirilemeyen Gereksinimler.....       | 47        |
| 7.2. Tasarımda Değişen Sınıflar / Diyagramlar.....                     | 48        |
| 7.3. Vazgeçtiğimiz / Değiştirdiğimiz Teknolojiler ve Nedenleri.....    | 49        |
| 7.4. Kullanmadığımız / Ertelediğimiz External API ve Kütüphaneler..... | 50        |

# 1. GİRİŞ

## 1.1. Raporun Amacı ve Bağlamı

Bu doküman, Yazılım Mühendisliği dersi kapsamında geliştirilen "Proof-Based Portfolio Platform" projesinin nihai durumunu, geliştirme sürecindeki mühendislik tecrübelerini ve teslim edilen ürünün teknik çıktısını belgelemek amacıyla hazırlanan final rapordur. Önceki süreçlerde sunulan analiz ve tasarım raporlarındaki teorik modelleri ve diyagramları tekrarlamak yerine; bu rapor doğrudan projenin çalışan son halinin neyi yapıp neyi yapamadığına, sistemin teknik çalışma prensiplerine, süreçte yapay zekanın nasıl konumlandırıldığına ve ekip içindeki somut iş bölümüne odaklanmaktadır.

## 1.2. Sistemin Amacı ve Çözüm Sunduğu Temel Problemler

Geleneksel özgeçmişler ve klasik portföy siteleri, yazılım geliştiricilerin yetkinliklerini yalnızca yazılı birer beyan olarak sunabilir. Bir adayın projesinin gerçekten çalışıp çalışmadığını, kod kalitesini veya kullanılan teknolojilerin derinliğini incelemek isteyen işverenler ile teknik liderler, her bir çalışma için harici platformlarda ek bir araştırma çabası sarf etmek zorundadır.

Proof-Based Portfolio Platform, bu doğrulanabilirlik ve teknik şeffaflık problemini kökten çözmek amacıyla geliştirilmiştir. Sistem, bento stili modern ve modüler bir görsel arayüzü, yazılım mühendisliğinin temel ihtiyacı olan "kanıt temelli" (proof-based) bir dijital kimlik modeliyle birleştirir. Platform üzerinde sergilenen her bir projenin; doğrudan açık kaynak kod erişimi (GitHub) ve çalışan bir canlı yazılım çıktısı (Demo URL) barındırması zorunludur. Böylece teknik çıktılar, beyan seviyesinden çıkarılarak somut ve anında doğrulanabilir birer kanıta dönüştürülür.

## 1.3. Projenin Nihai Kapsamı ve Modüler Sınırları

Proje, MVP (Minimum Uygulanabilir Ürün) hedeflerine tam uyumlu şekilde, birbiriyle entegre çalışan üç temel modül üzerinde tamamlanarak Vercel ve Neon PostgreSQL bulut altyapısı üzerinde canlıya alınmıştır:

- **Güvenli Kimlik Doğrulama Modülü:** Kullanıcıların e-posta ve şifre kombinasyonu ile sisteme kaydolmasını ve `iron-session` tabanlı, sunucu belleğinde durum tutmayan (stateless) şifreli çerezler vasıtasıyla güvenli oturum açmasını sağlar.
- **Profil ve Portföy Yapılandırma Modülü:** Kullanıcılara kendilerine özel benzersiz bir adres uzantısı (slug) tanımlama, kişisel bilgilerini (ad, biyografi, avatar) güncelleme ve portföylerini dış dünyaya yayınlama (`isPublished`) ya da taslak olarak gizleme imkanı tanır.
- **İçerik ve GitHub İçerik Aktarım Modülü:** Kullanıcıların manuel olarak proje ve sosyal medya linki eklemelerine, düzenlemelerine veya silmelerine olanak tanır. Modülün en kritik işlevi olan `GitHub Import` mekanizması sayesinde, kullanıcılar sadece GitHub kullanıcı adlarını girerek public kod depolarını tek bir senkron HTTP çağrısı ile portföylerine otomatik olarak aktarabilmektedir.



## 2. Proje Deneyimi

Proof-Based Portfolio Platform projesi boyunca analiz, tasarım ve implementasyon aşamalarını ayrı ayrı deneyimleme fırsatı bulduk. Bu süreçte yalnızca çalışan bir web uygulaması geliştirmeyi değil, bir yazılım projesinin gereksinimlerden başlayarak modele, modelden mimariye ve mimariden uygulanabilir bir ürüne nasıl dönüştürüldüğünü öğrendik. Proje süreci; gereksinim analizi, UML modelleme, mimari karar verme, teknoloji seçimi, güvenlik, performans ve bakım yapılabilirlik gibi konuların yazılım geliştirme sürecinin ayrılmaz parçaları olduğunu görmemizi sağladı.

### a. Analiz Aşaması

Analiz aşamasında öncelikle sistemin kullanıcılarını, temel kullanım senaryolarını ve proje kapsamını netleştirdik. Kullanıcıların sisteme kayıt olması, giriş yapması, profil ve portföy bilgilerini yönetmesi, proje ve bağlantı eklemesi, portföyünü yayınlaması ve dış ziyaretçilerin yayınlanmış portföyleri görüntülemesi gibi temel senaryoları belirledik. Bu süreçte bir sistemin doğru geliştirilebilmesi için önce hangi ihtiyaca cevap verdiğinin, hangi aktörlerle etkileşime girdiğinin ve hangi sınırlar içinde çalışacağını açıkça tanımlanması gerektiğini öğrendik.

Bu aşamada işlevsel gereksinimler, işlevsel olmayan gereksinimler ve sözde gereksinimler arasındaki farkı daha iyi kavradık. İşlevsel gereksinimler kapsamında kullanıcı yönetimi, portföy yönetimi, proje yönetimi, bağlantı yönetimi ve portföy görüntüleme işlemlerini tanımladık. İşlevsel olmayan gereksinimler kapsamında kullanılabilirlik, güvenlik, performans, uyumluluk ve genişletilebilirlik gibi kalite özelliklerini ele aldık. Sözde gereksinimlerde ise projenin **full-stack** bir web uygulaması olarak geliştirilmesi, ilk sürümde **e-posta/şifre tabanlı kimlik doğrulama** kullanılması ve veri modelinin **User**, **Portfolio**, **Project** ve **Link** temel nesneleri etrafında kurulması gibi teknik kısıtları belirledik.

Analiz aşamasında sistemi daha anlaşılır ve modellenabilir hale getirmek için **UML diyagramlarından** yararlandık. **Use Case diyagramı** ile kullanıcı ve ziyaretçi aktörlerinin sistemle nasıl etkileşime girdiğini gösterdik. **Class diyagramı** ile sistemdeki temel nesneleri, bu nesnelerin özelliklerini, metotlarını ve aralarındaki ilişkileri modelledik. **State diyagramı** ile özellikle **Portfolio**, **Project** ve **UserSession** gibi durum değişimi yaşayan nesnelerin yaşam döngüsünü inceledik. **Activity diyagramı** ile kayıt/giriş, portföy oluşturma-yayınlama ve proje/link yönetimi gibi iş akışlarını adım adım gösterdik. **Sequence diyagramı** ile kayıt, giriş, portföy düzenleme, yayınlama, public portföy görüntüleme ve dış veri kaynağından proje içe aktarma gibi senaryolarda nesneler arasındaki mesajlaşma sırasını analiz ettik. Bu diyagramlar, sistemi kodlamadan önce daha somut, anlaşılır ve kontrol edilebilir hale getirmemize yardımcı oldu.

### b. Tasarım Aşaması

Tasarım aşamasında analizde belirlediğimiz gereksinimleri uygulanabilir bir yazılım mimarisine dönüştürdük. Bu aşamada en önemli kazanımımız, bir sistemin yalnızca ekranlardan ve

veritabanından oluşmadığını; katmanlar, sorumluluklar, bağımlılıklar, güvenlik, performans ve bakım yapılabirlik gibi unsurlarla birlikte tasarlanması gerektiğini görmek oldu.

Projenin mimarisinde **BCE (Boundary-Control-Entity)** yaklaşımını benimsedik. **Boundary** katmanında kullanıcı arayüzü bileşenleri, **Next.js** sayfaları, form yapıları ve **API route handler'ları** yer aldı. **Control** katmanında **AuthController**, **PortfolioController**, **ProjectController** ve **LinkController** gibi iş mantığını yöneten sınıflar tasarlandı. **Entity** katmanında ise **User**, **Portfolio**, **Project** ve **Link** veri yapıları ile PostgreSQL veritabanı şeması konumlandırıldı. Bu ayırım sayesinde arayüz, iş mantığı ve veri erişimi arasında net bir sorumluluk paylaşımı kurmayı öğrendik.

Tasarım sürecinde bazı mimari kararları bilinçli şekilde verdik. Başta ayrı frontend ve backend yapısı ile Python tabanlı bir backend alternatifi değerlendirildiyse de, MVP kapsamı için iki ayrı deploy süreci, CORS ayarları ve farklı teknolojiler arasında tip senkronizasyonu ek yük oluşturacağı için **full-stack Next.js** mimarisinde karar kıldık. Böylece frontend, API route'ları ve iş mantığı tek proje içinde daha sade ve yönetilebilir hale geldi. Veritabanı için Neon üzerinde PostgreSQL kullanmaya karar verdik. Veri erişiminde **Prisma** yerine **Drizzle ORM** tercih edildi; çünkü Drizzle'ın Neon serverless yapısı ile doğrudan uyumlu olması, daha küçük paket boyutuna sahip olması ve **TypeScript** ile tip güvenli sorgular yazmaya olanak sağlaması bizim mimarimize daha uygun görüldü. Kimlik doğrulama tarafında **NextAuth** ve **GitHub OAuth** yerine **iron-session** kullanıldı; çünkü MVP kapsamında sosyal girişe ihtiyaç duymadan, e-posta ve şifre tabanlı daha sade bir oturum yönetimi yeterliydi.

Performans açısından public portföy sayfalarında **SSR** yerine **SSG/ISR** yaklaşımını kullanmayı öğrendik. Bu karar sayesinde herkes tarafından görüntülenen portföy sayfalarının her istekte veritabanına gitmesi yerine, CDN üzerinden hızlı sunulması hedeflendi. Ancak statik sayfalarda güncel veri problemi oluşabileceği için **ISR** ve **revalidatePath()** mekanizmasının önemini de gördük. Güvenlik açısından kullanıcı şifrelerinin **bcrypt** ile hashlenmesi, oturumların **iron-session** ile şifrelenmiş cookie üzerinden yönetilmesi ve verilerin **Zod** şemaları ile doğrulanması tasarımın temel parçaları haline getirildi. Arayüz tarafında **CSS Modules** yerine **Tailwind CSS v4** ve **shadcn/ui** tercih edilerek sade, tutarlı ve yeniden kullanılabilir bileşen yapısı planlandı.

Tasarım aşamasında analizdeki sınıf yapısını daha uygulanabilir hale getirdik. Genel arayüz yapıları; **LoginForm**, **RegisterForm**, **ProjectModal**, **LinkModal**, **ProfileEditor**, **ThemeSelector**, **BentoGrid**, **ProfileCard**, **ProjectCard**, **LinkCard** ve **ThemeProvider** gibi gerçek React bileşenlerine ayrıştırıldı. Ayrıca tasarım seviyesinde deployment diyagramı, paket/katman diyagramı ve ER modeliyle sistemin çalışma ortamı, paket yapısı ve veritabanı ilişkileri daha ayrıntılı biçimde gösterildi. Bu aşama bize, analiz diyagramlarının doğrudan kodlanmadığını; tasarım aşamasında daha somut, uygulanabilir ve teknolojiye uygun hale getirilmesi gerektiğini öğretti.

### c. Implementasyon Aşaması

Implementasyon aşamasında tasarımda belirlenen mimari kararları gerçek kod yapısına dönüştürmeyi öğrendik. Proje geliştirilirken **Next.js 16 App Router**, **React 19**, **TypeScript**,

Tailwind CSS v4, shadcn/ui, PostgreSQL/Neon, Drizzle ORM, iron-session, bcrypt, Zod, React Hook Form, Resend, Vercel serverless altyapısı ve Vitest gibi teknolojiler üzerinde çalıştık. Bu teknolojiler sayesinde modern bir full-stack web uygulamasında frontend, backend, veritabanı, kimlik doğrulama, doğrulama kuralları, API yapısı, e-posta gönderimi, test ve dağıtım süreçlerinin nasıl birlikte çalıştığını deneyimledik.

Kodlama sürecinde kullanıcı kayıt/giriş işlemleri, oturum yönetimi, parola sıfırlama akışı, portföy oluşturma, profil düzenleme, proje ve bağlantı ekleme/düzenleme/silme, portföy yayınlama/gizleme, sınırlı tema seçimi ve public portföy görüntüleme gibi temel işlevlerin nasıl geliştirileceğini öğrendik. Form işlemlerinde React Hook Form ve Zod kullanarak kullanıcıdan gelen verileri hem istemci hem de sunucu tarafında doğrulamanın önemini gördük. Veritabanı tarafında Drizzle ORM ile User, Portfolio, Project, Link ve parola sıfırlama süreci için gerekli token yapısı arasındaki ilişkileri yönetmeyi, cascade davranışlarını ve tip güvenli sorguları kullanmayı öğrendik.

Implementasyon sürecinde tasarımın kodlama sırasında değişebileceğini de deneyimledik. Profil güncellemenin portföy değil kullanıcı işlemi olduğunu fark ederek UserController gibi ayrı bir kontrol sınıfına ihtiyaç duyulabileceğini gördük. Dashboard tarafında oturum açmış kullanıcının kendi portföyünü yükleyebilmesi için ek metotlara ihtiyaç olduğunu öğrendik. Ayrıca parola sıfırlama akışı için başlangıçtaki ER modelinde bulunmayan ek bir token tablosu gerektiğini fark ettik. Bu değişiklikler, tasarımın önemli bir başlangıç noktası olduğunu; fakat implementasyon sırasında gerçek ihtiyaçlara göre kontrollü şekilde geliştirilebileceğini gösterdi.

Implementasyon sürecinde bazı teknik zorluklarla da karşılaştık. Serverless mimaride cold start problemi, kullanılan kütüphanelerin boyutunun ve veritabanı bağlantı stratejisinin performansı etkileyebileceğini gösterdi. Bu nedenle Drizzle ORM ve Neon serverless yapısının tercih edilmesi uygulamanın daha hafif ve uygun çalışmasına katkı sağladı. Ayrıca public portföy sayfalarında SSG/ISR kullanılırken, kullanıcı portföyünü güncellediğinde ziyaretçilerin eski veriyi görmemesi için revalidatePath() mekanizmasının doğru kullanılmasının gerekli olduğunu öğrendik. Middleware tarafında da oturum doğrulamasının her durumda aynı yerde yapılmasının performans ve çalışma ortamı açısından uygun olmayabileceğini, bu nedenle bazı kontrollerin Controller seviyesinde ele alınmasının daha doğru olabileceğini gördük.

GitHub'dan otomatik proje içe aktarma özelliği analiz ve tasarım aşamalarında ele alınmış olsa da, implementasyon sırasında GitHub API rate-limit yönetimi ve dönen verilerin Project modeline eşlenmesi gibi konuların beklenenden daha kapsamlı olduğu görüldü. Bu nedenle bu özellik MVP kapsamında tamamlanan temel işlevler arasında yer almadı. Bu deneyim bize, bir özelliğin analiz ve tasarımda yer almasının, implementasyon aşamasında zaman, karmaşıklık ve proje kapsamı açısından yeniden değerlendirilmesi gerekebileceğini gösterdi.

Bu proje sonunda yazılım mühendisliğine bakış açımız önemli ölçüde değişti. Başlangıçta yazılım geliştirmeyi daha çok kod yazma süreci olarak görürken, proje ilerledikçe başarılı bir sistem için analiz, tasarım, mimari kararlar, veri modeli, güvenlik, performans, test edilebilirlik ve

bakım yapılabilirlik gibi konuların en az kodlama kadar önemli olduğunu gördük. UML diyagramları sayesinde sistemi kodlamadan önce düşünmenin ve modellemenin, implementasyon sırasında karşılaşılabilecek belirsizlikleri azalttığını fark ettik. Ayrıca her teknolojinin avantajları ve dezavantajları olduğunu; doğru seçimin yalnızca popüler olanı kullanmak değil, projenin kapsamına, zamanına, hedeflerine ve teknik ihtiyaçlarına en uygun çözümü seçmek olduğunu öğrendik.

### 3. Yapay Zeka Kullanımı

Bu projenin geliştirme sürecinde üretken yapay zeka (GenAI) araçları, süreci hızlandıran ve kod kalitesini optimize eden birer **"tasarım ve kod gözden geçirme (code/design review) ortağı"** olarak entegre edilmiştir. Geliştirme sürecinde yapay zeka araçlarının kullanımı gizlenmemiş, aksine bir mühendislik yetkinliği olarak somut çıktılar ve sürüm kontrol (Git) geçmişi üzerinden şeffaf bir şekilde belgelenmiştir.

#### 3.1. Kullanılan Yapay Zeka Araçları ve Kapsamı

Projenin sadece **kimlik doğrulama (auth)**, **oturum yönetimi (session)** ve **temel altyapı kurulumu** aşamalarında yapay zeka desteğine başvurulmuştur. Kodun diğer tüm modülleri (Portföy yönetimi, proje ve link CRUD işlemleri vb.) tamamen manuel olarak geliştirilmiş olup, yapay zekanın katkısı Git commit geçmişiyle sınırlı ve izole tutulmuştur. Projede kullanılan LLM (Büyük Dil Modeli) tabanlı araçlar şunlardır:

- **Claude 4.6 Opus / Gemini 3.1 Pro:** Mimari tasarım kararları, katmanlı mimari (BCE) sınırlarının belirlenmesi ve UML diyagram yapılarının mantıksal kontrolü süreçlerinde kullanılmıştır.
- **GitHub Copilot:** Kodlama esnasında **iron-session** entegrasyonu, TypeScript tip tanımlamalarının otomatik tamamlanması ve Zod şemalarının hızlıca yazılması aşamalarında yardımcı olmuştur.

#### 3.2. Sürüm Kontrolü ve Eş-Geliştirme (Co-Authoring) Kanıtları

Yapay zeka araçları projede körü körüne bir kod üreticisi olarak kullanılmamış, belirli iş paketlerinde **"Pair Programming" (Eşli Programlama)** mantığıyla sürece dahil edilmiştir. Bu durum, projenin GitHub commit geçmişinde yapay zekanın geliştirme ortağı olarak imzalanmasıyla (**co-committed with claude**) somut olarak kanıtlanmıştır:

- **docs: replace project PDFs with living MD documents:** Proje dokümantasyonunun statik PDF formatından, sürekli güncellenen dinamik Markdown (**.md**) formatına taşınmasında AI'nın yapılandırma yeteneğinden yararlanılmıştır.
- **feat(auth): add AuthController with bcrypt and reset flow:** İş mantığı (Control) katmanı inşa edilirken, şifreleme algoritmaları ve parola sıfırlama akışları AI ile birlikte kurgulanmıştır.
- **feat(api): add auth route handlers:** API uç noktalarının (Boundary) kurulmasında ve request body verilerinin Zod şemaları ile sarmalanmasında AI asistanlığı kullanılmıştır.

- `feat(middleware): add session-based route protection`: Next.js `middleware.ts` dosyasında, gelen isteklerin oturum durumuna göre ayıklanması ve korumalı rotalara (`/dashboard`) yönlendirilmesi süreçleri optimize edilmiştir.

### 3.3. Yapay Zekanın Katkısı ve Mühendislik Denetimi

Yapay zekanın sunduğu öneriler hiçbir zaman doğrudan (kopyala-yapıştır yöntemiyle) projeye dahil edilmemiştir. Üretilen her mimari öneri, projenin kısıtlamalarına ve teknoloji yığını parametrelerine göre süzgeçten geçirilmiş, anlaşılıp uyarlanarak sisteme entegre edilmiştir. Geliştirme sürecinde yapay zekanın mimari kısıtları ihlal eden veya güvenlik açığı barındıran kritik hatalar yaptığı saptanmış ve bu hatalar ekibimiz tarafından manuel olarak düzeltilmiştir:

#### Somut Hata 1: Mimari Kısıtların İhlali (Tablo Uydurma)

Görseldeki `feat(db): add password reset token table` adımı AI, şifre sıfırlama akışını çözmek için veritabanında bağımsız bir `password_reset_tokens` tablosu açmayı önermiş ve kodu bu şekilde yönlendirmiştir.

Ancak projenin pseudo gereksinimlerinde yer alan *"Veri modeli sadece User, Portfolio, Project, Link üzerinden — ekstra tablo açılmayacak"* kuralını korumak adına bu tasarıma **müdahale edilmiştir**. Ekstra tablo yaklaşımı elenmiş; token yapısı `User` tablosuna `resetToken` ve `resetTokenExpiresAt` alanları olarak yedirilerek kısıtlamalara tam uyum sağlanmıştır.

#### Somut Hata 2: App Router Yerine Eski Pages Router Mimarisi Önerisi

Oturum yönetimi tasarımı sırasında AI modeli, Next.js App Router mimarisini göz ardı ederek Pages Router'a uygun olan ve parametre olarak HTTP istek/yanıt nesnelerini alan `getSession(req, res)` şeklinde eski bir arayüz önermiştir.

Ekibimiz, App Router dünyasında `next/headers` altındaki `cookies()` context'inin kullanılması gerektiğini fark etmiş, tasarımı parametresiz ve doğrudan sunucu tarafında header tüketen `getSession(): Promise<IronSession>` formuna revize ederek teknik tutarsızlığı gidermiştir.

#### Somut Hata 3: Hesap Tespiti (Account Enumeration) Güvenlik Açığı

Şifre sıfırlama akışında yapay zeka, iç mantıkta fırlatılan `UserNotFound` (Kullanıcı Bulunamadı) hatasını doğrudan API katmanına ve HTTP response'a yansıtacak bir kod bloğu üretmiştir.

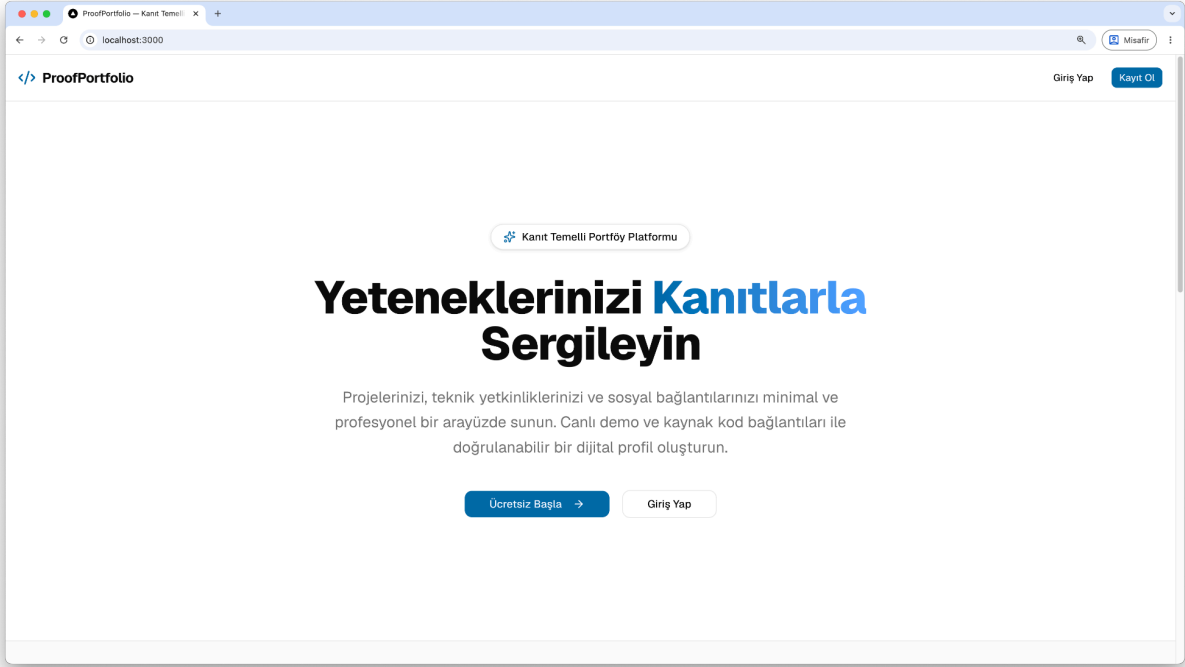
Bu durumun bir saldırganın sistemdeki kayıtlı e-postaları tahmin etmesine yol açacağını (Account Enumeration) fark eden ekibimiz, kodu reddetmiştir. İş akışı, iç mantıkta hata oluşsa bile dışarıya her durumda başarılı mesajı dönen idempotent ve jenerik bir yapıya dönüştürülmüştür.

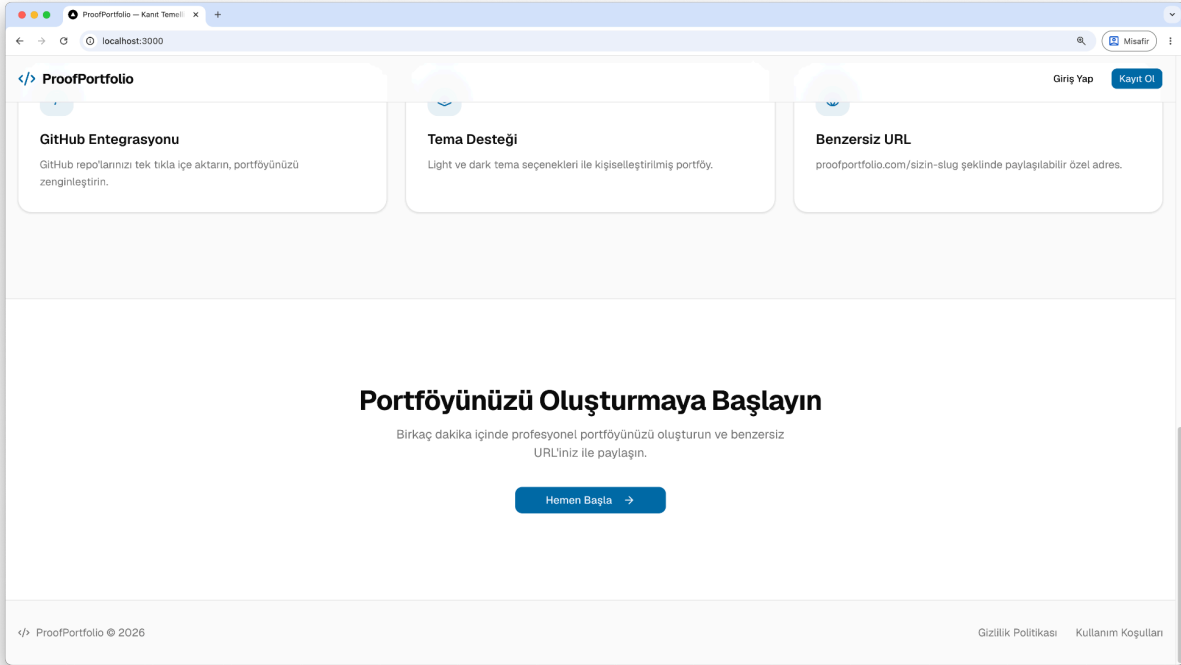
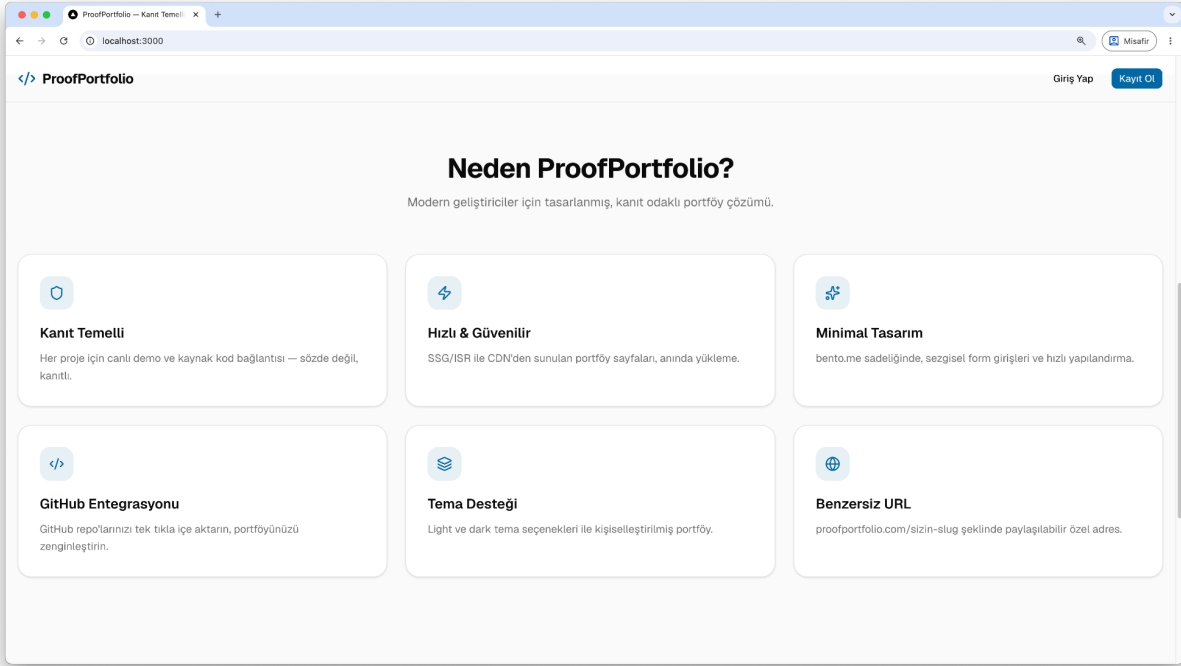
## 4. Kullanıcı Kılavuzu

Bu bölümde sistemdeki tüm ekranlar ve özellikler, kullanıcı akışına göre sırayla anlatılmaktadır. Her ekran için kısa bir açıklama ve ekran görüntüsü referansı verilmiştir.

### 4.1. Ana Sayfa (Landing Page) — /

**Aktör:** Ziyaretçi (Guest)





Platformun giriş sayfasıdır. Kullanıcıyı karşılayan hero bölümünde:

- Platformun sloganı: *"Yeteneklerinizi Kanıtlarla Sergileyin"*
- Kısa açıklama metni
- **"Ücretsiz Başla"** butonu (</register> sayfasına yönlendirir)



- **"Giriş Yap"** butonu (</login> sayfasına yönlendirir)

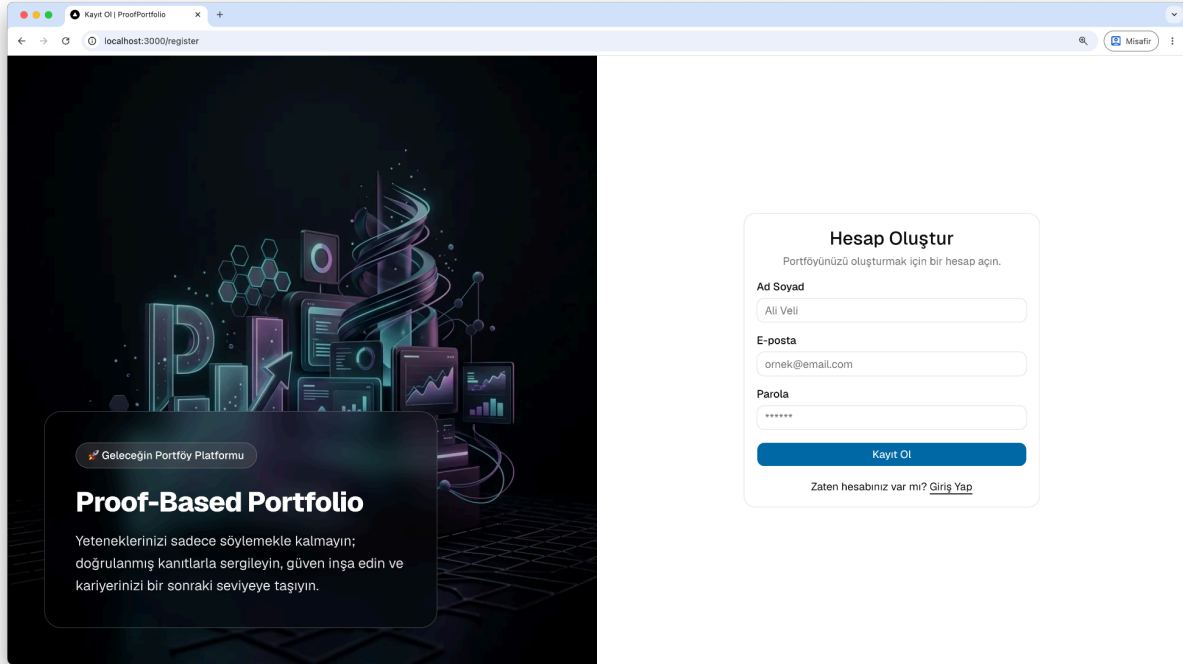
Alt kısımda 6 adet özellik kartı bulunur:

1. **Kanıt Temelli** — Her proje için canlı demo ve kaynak kod bağlantısı
2. **Hızlı & Güvenilir** — SSG/ISR ile CDN'den sunulan portföy sayfaları
3. **Minimal Tasarım** — bento.me sadeliğinde, sezgisel form girişleri
4. **GitHub Entegrasyonu** — GitHub repo'larını tek tıkla içe aktarma
5. **Tema Desteği** — Light ve dark tema seçenekleri
6. **Benzersiz URL** — [proofportfolio.com/sizin-slug](https://proofportfolio.com/sizin-slug) şeklinde özel adres

Footer'da **Gizlilik Politikası** ve **Kullanım Koşulları** bağlantıları yer alır.

## 4.2. Kayıt Ol (Register) — </register>

**Aktör:** Ziyaretçi (Guest)



Split-panel bir düzen kullanılır:

- **Sol panel:** Arka plan görseli üzerinde platform tanıtım metni (glassmorphism kart)
- **Sağ panel:** Kayıt formu

Form alanları:



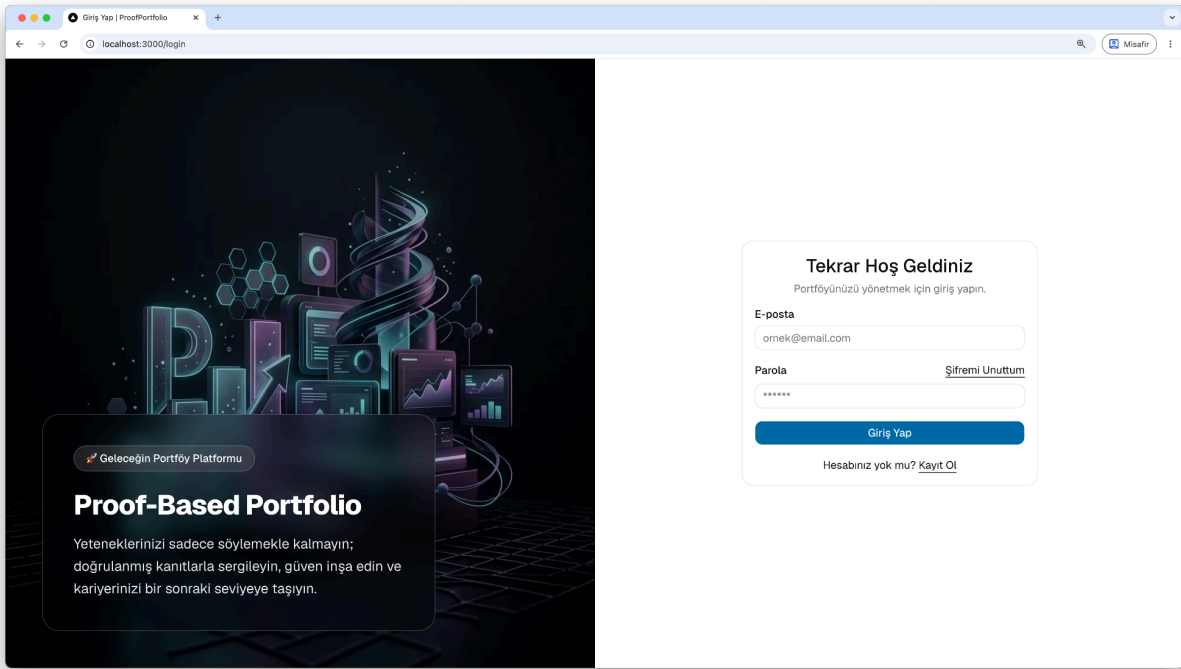
- **E-posta** (`email`) — Zod `.email()` doğrulaması, küçük harfe normalize
- **Ad Soyad** (`fullName`) — İsteğe bağlı, maksimum 100 karakter
- **Parola** (`password`) — Minimum 8 karakter, bcrypt ile hash'lenir

Alt bağlantı: "Zaten hesabınız var mı? Giriş Yap"

Form gönderildiğinde `POST /api/auth/register` çağrılır. Başarılı kayıt sonrası otomatik giriş yapılır (`iron-session` cookie oluşturulur) ve kullanıcı Dashboard'a yönlendirilir. Hata durumunda toast bildirimi gösterilir (ör. "Bu e-posta zaten kayıtlı").

### 4.3. Giriş Yap (Login) — `/login`

**Aktör:** Ziyaretçi (Guest)



Kayıt sayfasıyla aynı split-panel düzeni kullanılır.

Form alanları:

- **E-posta** (`email`)
- **Parola** (`password`)

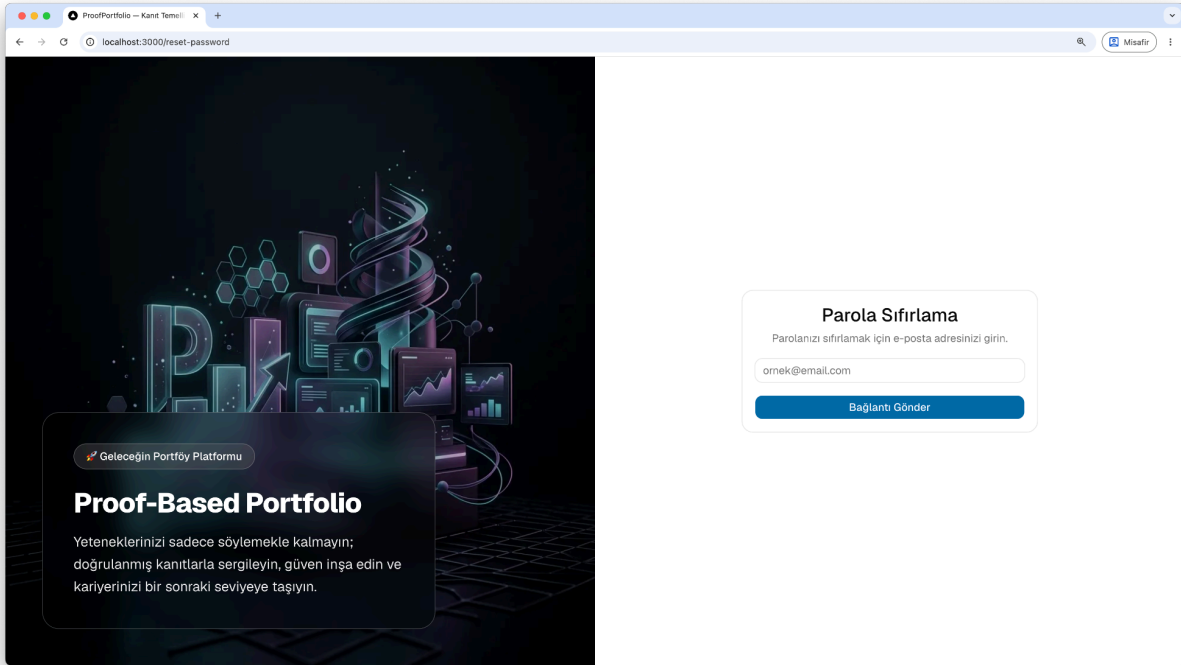
Güvenlik: Hatalı giriş denemelerinde kullanıcıya *"Email or password incorrect"* şeklinde jenerik hata mesajı gösterilir. Hesap numaralandırmasını (account enumeration) önlemek için e-posta adresinin kayıtlı olup olmadığı bilgisi sızdırılmaz.

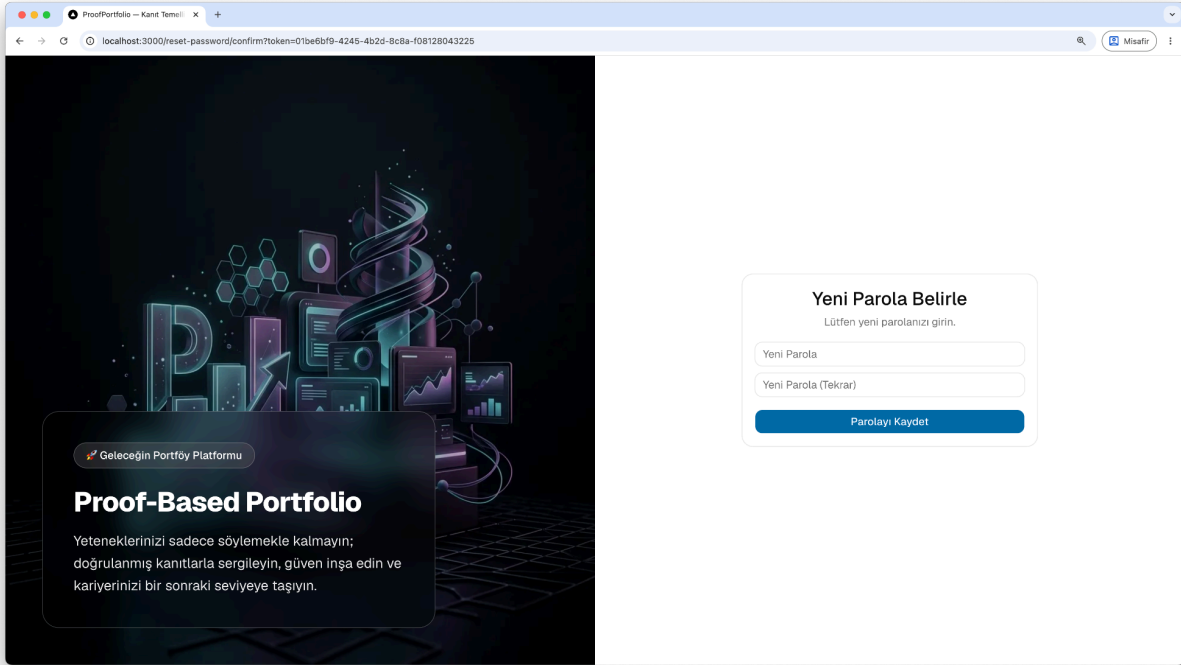
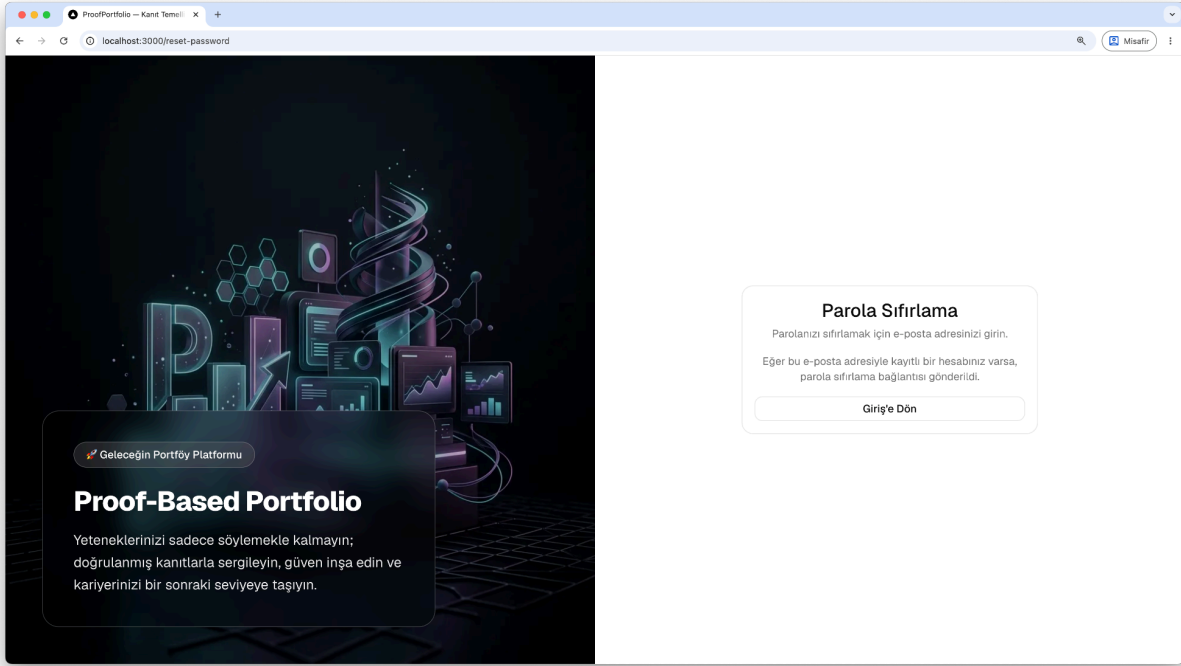
Alt bağlantı: *"Hesabınız yok mu? Kayıt Ol"* ve üst kısımda **"Parolanızı mı unuttunuz?"** bağlantısı.

Başarılı girişte **iron-session** cookie oluşturulur (7 gün TTL, HttpOnly, Secure) ve Dashboard'a yönlendirilir.

#### 4.4. Parola Sıfırlama — /reset-password

**Aktör:** Ziyaretçi (Guest)





**Adım 1 — Talep (/reset-password):** Kullanıcı e-posta adresini girer. Güvenlik gereği, hesap var veya yok fark etmeksizin her zaman "Eğer bu e-posta adresiyle kayıtlı bir hesabınız varsa, parola sıfırlama bağlantısı gönderildi." mesajı gösterilir. Arka planda:

- Tek kullanımlık token üretilir (cuid + hash)

- Token 1 saat TTL ile veritabanına kaydedilir
- Resend API üzerinden e-posta gönderilir (API key yoksa konsola log)

**Adım 2 — Onay (/reset-password/confirm?token=...):** E-postadaki linke tıklayan kullanıcıdan yeni parola (ve tekrarı) istenir. Token doğrulaması yapılır, geçerliyse:

- Yeni parola bcrypt ile hash'lenir
- Token geçersiz kılınır
- Tüm aktif session'lar sonlandırılır
- Kullanıcı login sayfasına yönlendirilir

## 4.5. Dashboard — Portföy Oluşturma (Onboarding)

**Aktör:** Kullanıcı (User) — İlk giriş

Kayıt Ol | ProofPortfolio

localhost:3000/register

Misafir

**Hesap Oluştur**  
Portföyünüzü oluşturmak için bir hesap açın.

Ad Soyad  
Mustafa Pekiririci

E-posta  
pekkiriscim@gmail.com

Parola  
.....

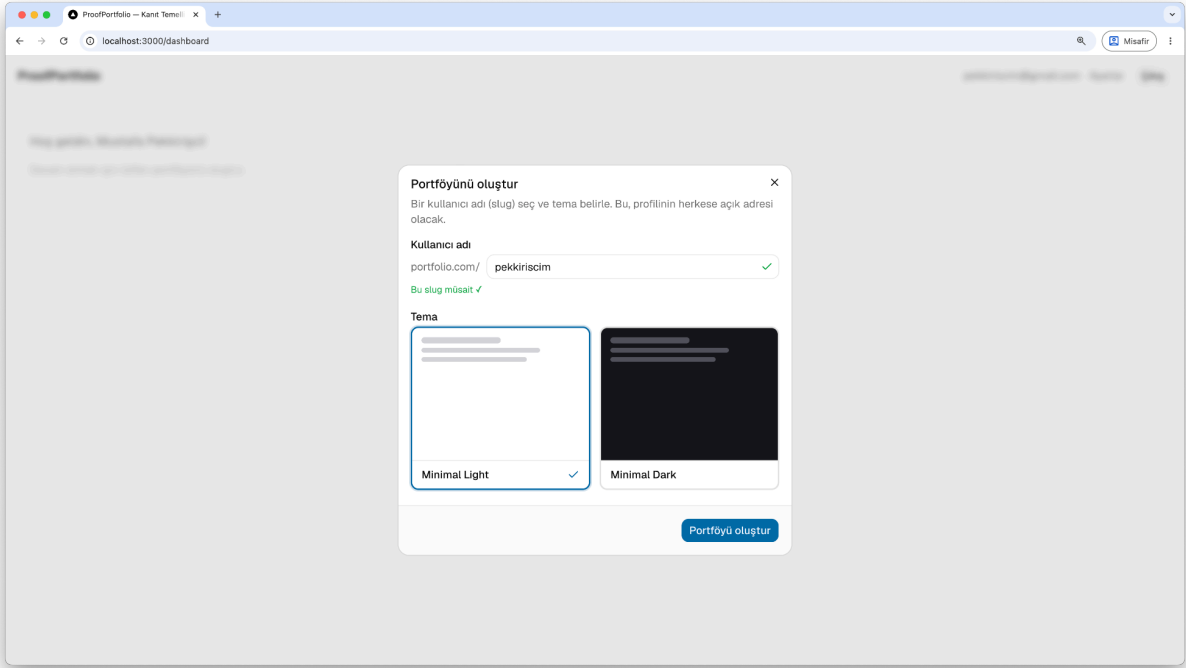
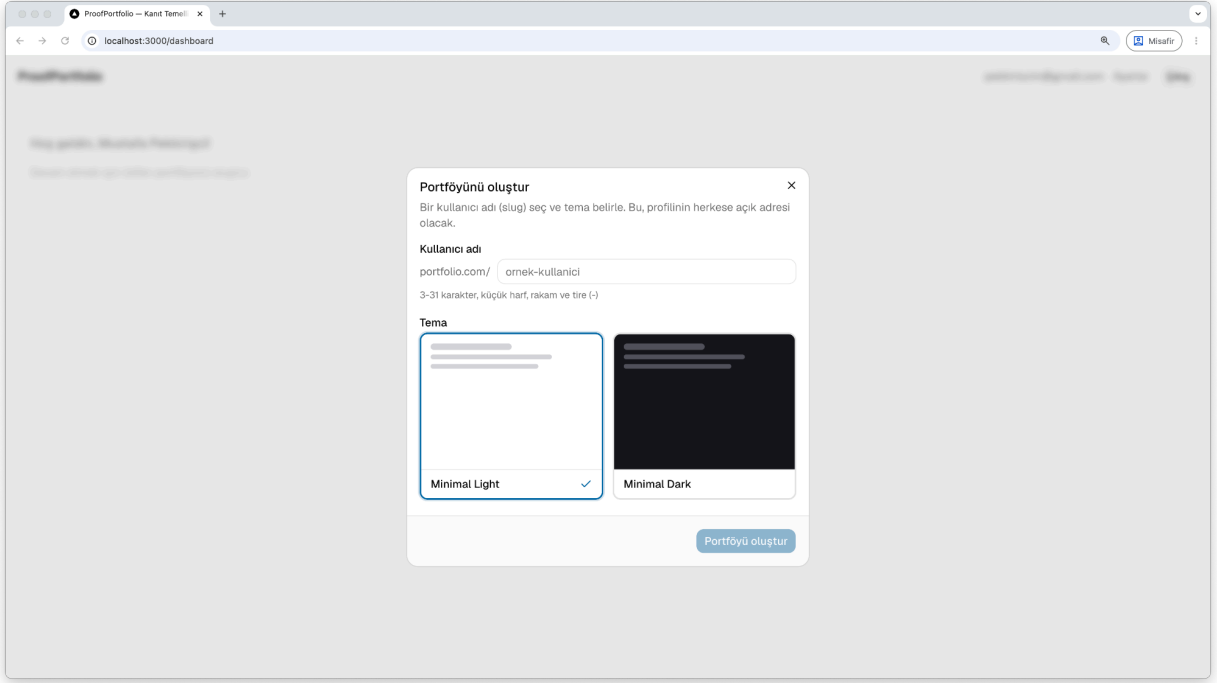
[Kayıt Ol](#)

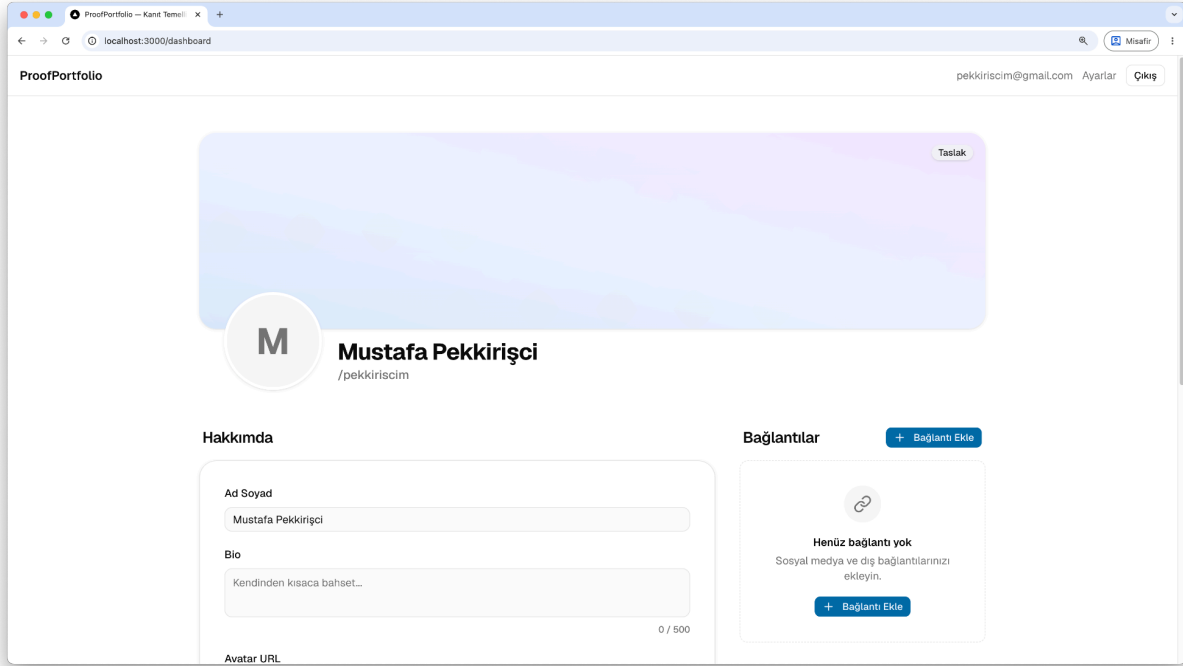
Zaten hesabınız var mı? [Giriş Yap](#)

**Geleceğin Portföy Platformu**

**Proof-Based Portfolio**

Yeteneklerinizi sadece söylemekle kalmayın;  
doğrulanmış kanıtlarla sergileyin, güven inşa edin ve  
kariyerinizi bir sonraki seviyeye taşıyın.





Kullanıcı kayıt olduktan sonra Dashboard'a yönlendirildiğinde, henüz bir portföyü yoksa **Onboarding Modal** otomatik olarak açılır. Modal kapatılamaz (kullanıcı portföy oluşturmadan devam edemez).

Modal içeriği:

- **Slug (Kullanıcı Adı):** `portfolio.com/ornek-kullanici` formatında özel URL. 3-31 karakter, küçük harf, rakam ve tire (-) içerebilir.
  - Yazım sırasında 400ms debounce ile canlı müsaitlik kontrolü (`GET /api/portfolio/check-slug?slug=...`)
  - Durum göstergeleri: ✓ müsait, ✗ kullanımda, ⌚ kontrol ediliyor
  - Rezerve kelimeler (api, dashboard, login, admin vb.) engellenir
- **Tema Seçimi:** "Minimal Light" ve "Minimal Dark" olmak üzere iki tema kartı. Seçilen tema vurgulanır.

"Portföyü oluştur" butonuna basıldığında `POST /api/portfolio` çağrılır. Başarılı olursa dashboard ana sayfası yenilenir ve onboarding modal kaybolur.

## 4.6. Dashboard — Ana Sayfa (/dashboard)

**Aktör:** Kullanıcı (User) — Portföy sahibi

The screenshot shows the dashboard for a user named Mustafa Pekirirci. The header includes the site name 'ProofPortfolio', the user's email 'pekirircim@gmail.com', and links for 'Ayarlar' (Settings) and 'Çıkış' (Logout). The main content area features a large profile picture placeholder with a 'Taslak' (Draft) label. Below the profile picture, the user's name 'Mustafa Pekirirci' and username '/pekirircim' are displayed. The dashboard is divided into two main sections: 'Hakkımda' (About Me) and 'Bağlantılar' (Links). The 'Hakkımda' section contains input fields for 'Ad Soyad' (Name Surname), 'Bio', and 'Avatar URL', along with a 'Kaydet' (Save) button. The 'Bağlantılar' section has a 'Henüz bağlantı yok' (No links yet) message and a '+ Bağlantı Ekle' (Add Link) button.

This screenshot shows the 'Proje' (Project) section of the dashboard. The 'Hakkımda' section is partially visible at the top. The 'Proje' section includes a 'GitHub Import' button and a text input field for 'kullanıcı-adi' (username). Below this, there is a 'Henüz proje yok' (No projects yet) message and a '+ Proje Ekle' (Add Project) button. The 'Yönetim' (Management) section is also visible, showing 'Yayın durumu' (Publish status) with a toggle switch and 'Tema' (Theme) options: 'Minimal Light' (selected) and 'Minimal Dark'.

Dashboard üç ana bölümden oluşur:

### Üst Kısım — Hero Kart:

- Kapak görseli (gradient placeholder)
- Avatar (kullanıcı resmi veya baş harf fallback)
- Kullanıcı adı ve slug gösterimi
- Yayın durumu rozeti: **"Yayında"** (yeşil) veya **"Taslak"** (gri)
- "Görüntüle" butonu (sadece yayındayken görünür, public portföyü yeni sekmede açar)

### Sol Kolon (Ana İçerik):

- **Hakkımda** — Profil düzenleme alanı
- **Projeler** — Proje grid listesi + Ekle butonu + GitHub Import

### Sağ Kolon (Sidebar):

- **Bağlantılar** — Sosyal medya bağlantı listesi + Ekle butonu
- **Yönetim** — Yayınlama toggle + Tema seçici

### Üst menüde:

- Kullanıcı e-posta adresi
- **Ayarlar** bağlantısı
- **Çıkış Yap** butonu



## 4.7. Dashboard — Profil Düzenleme

**Aktör:** Kullanıcı (User)

The screenshot shows a web browser window with the URL `localhost:3000/dashboard`. The page displays a user profile for **Mustafa Pekirisci** with the username `/pekkiriscim`. The profile includes a circular avatar and a header banner. Below the header, there are two main sections: **Hakkımda** (About Me) and **Bağlantılar** (Links).

**Hakkımda** section contains three input fields:

- Ad Soyad** (Full Name): `Mustafa Pekirisci`
- Bio**: `Computer Engineering Student` (with a character count of 28 / 500)
- Avatar URL**: `https://avatars.githubusercontent.com/u/121763536?v=4`

A **Kaydet** (Save) button is located at the bottom right of the **Hakkımda** section.

**Bağlantılar** section has a **+ Bağlantı Ekle** (Add Link) button. Below it, a message states: **Henüz bağlantı yok** (No links yet). It says: `Sosyal medya ve diğer bağlantılarınızı ekleyin.` (Add your social media and other links). There is another **+ Bağlantı Ekle** button.

**Yönetim** (Management) section has a **Yayın durumu** (Publish status) toggle switch. The text below it says: `Açıkken portföyün herkese açık adreste görünür.` (When open, the portfolio is visible to everyone at an open address).

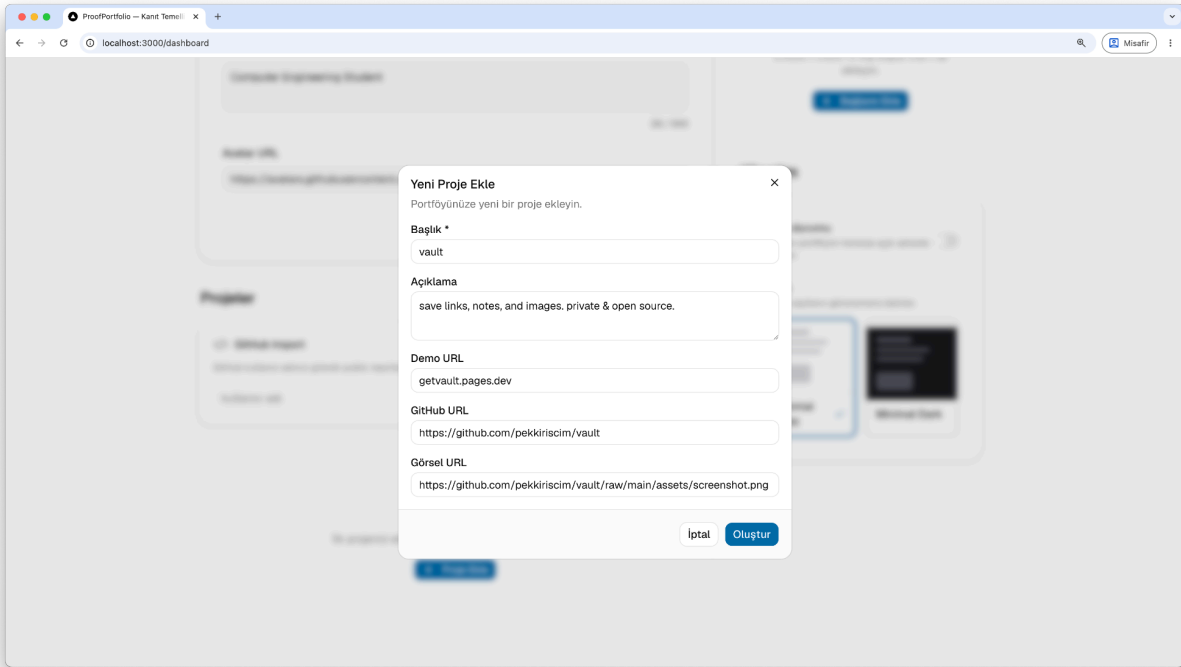
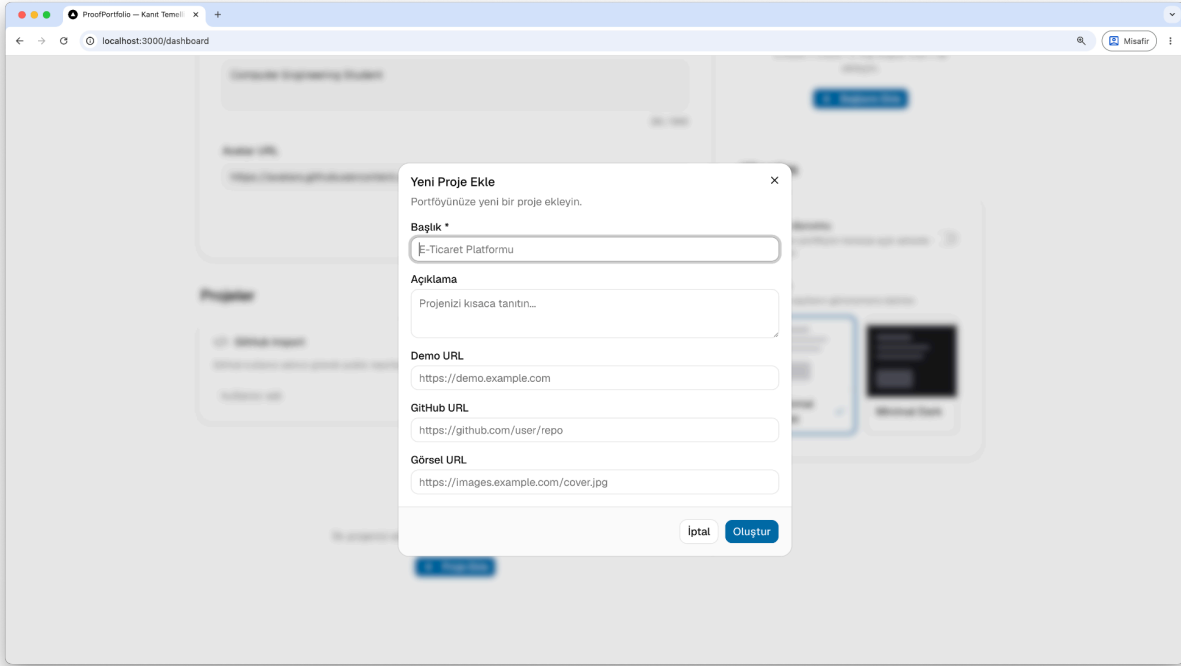
"Hakkımda" kartı içinde yer alan form:

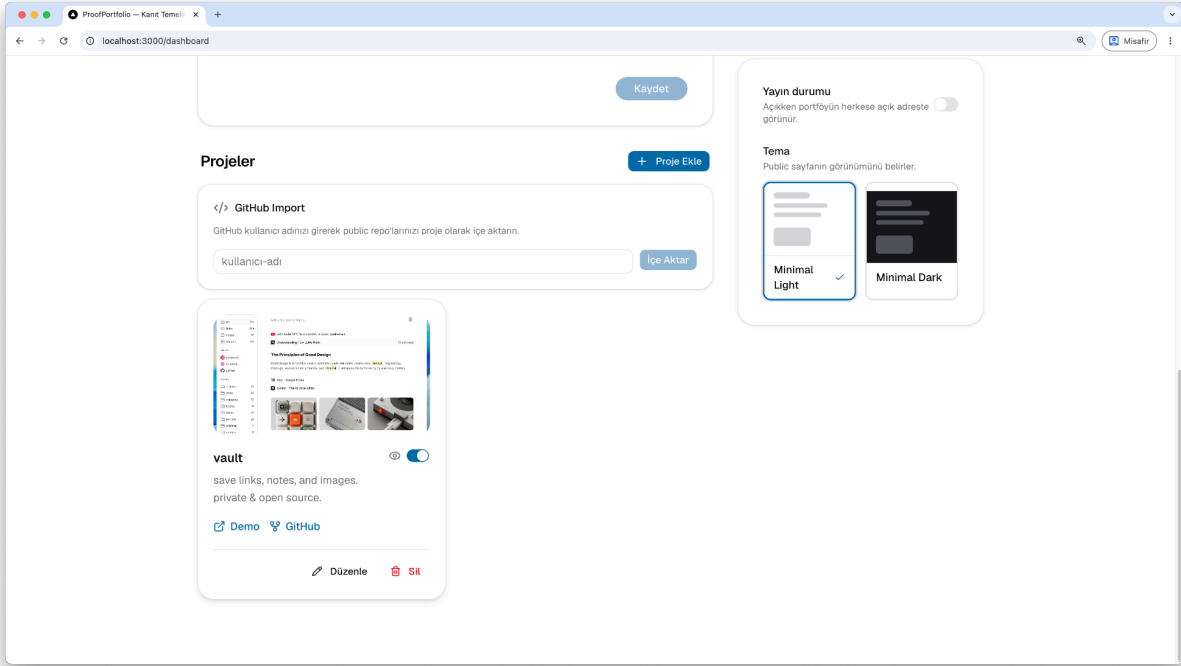
- **Ad Soyad** (`fullName`) — Maksimum 100 karakter
- **Biyografi** (`bio`) — Maksimum 500 karakter, altında canlı karakter sayacı
- **Avatar URL** (`avatarUrl`) — Harici bir resim bağlantısı (MVP'de dosya yükleme yok)

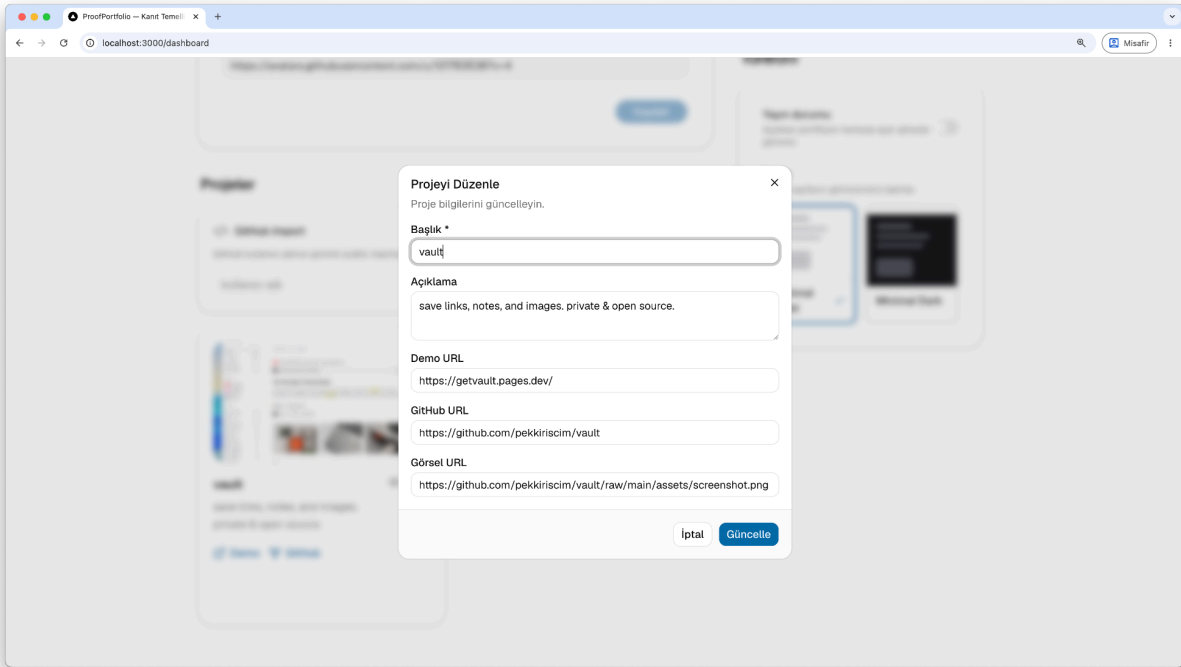
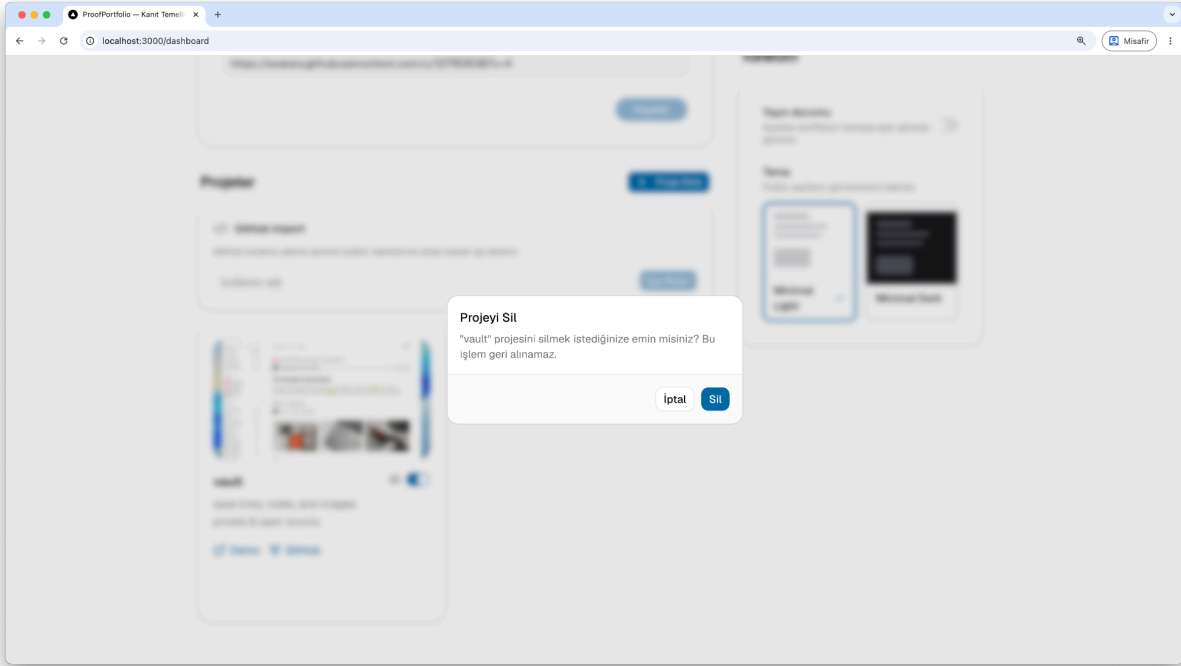
Her alan için optimistic UI kullanılır: kullanıcı kaydet butonuna bastığında UI hemen güncellenir, `PATCH /api/user/profile` isteği arka planda gider. Hata durumunda eski değerlere geri dönülür ve toast ile hata bildirilir.

## 4.8. Dashboard — Proje Yönetimi

**Aktör:** Kullanıcı (User)







## Proje Listesi:

- Grid layout, createdAt DESC sıralama
- Her proje kartında: başlık, açıklama (kırpılmış), demo/github link ikonları

- Sağ üstte üç buton: **Görünürlük toggle** (Switch), **Düzenle** (kalem ikonu), **Sil** (çöp kutusu)
- Boş durumda: *"Henüz proje eklenmemiş. İlk projenizi ekleyin →"*

#### Proje Ekleme/Düzenleme Modalı (Dialog):

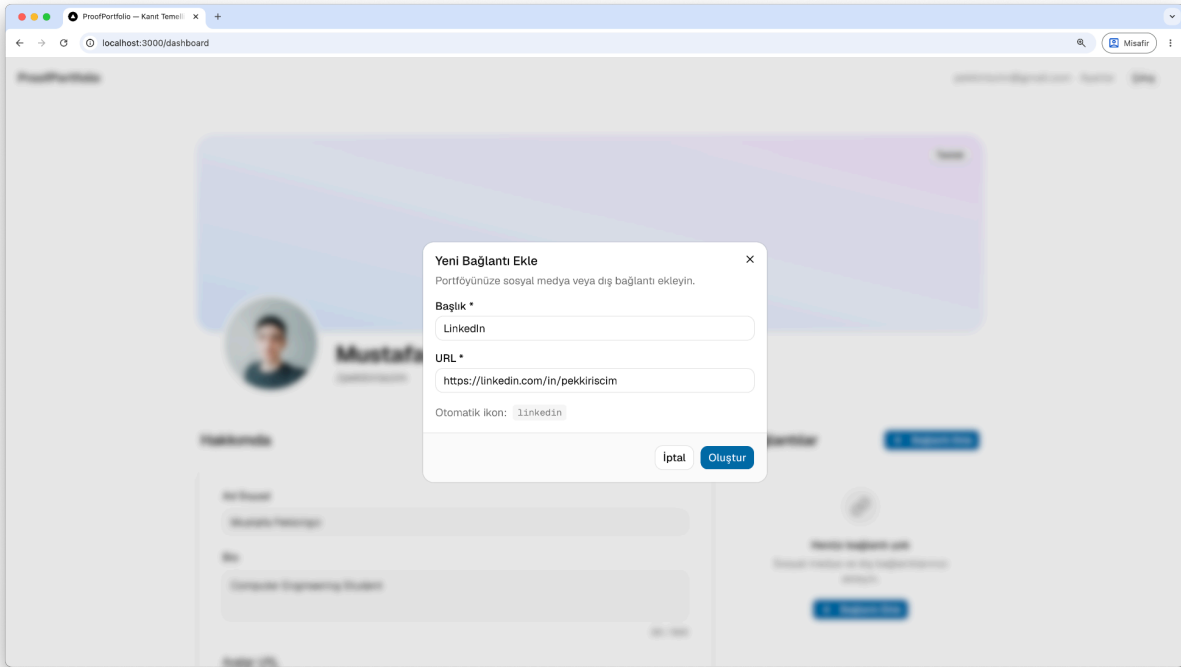
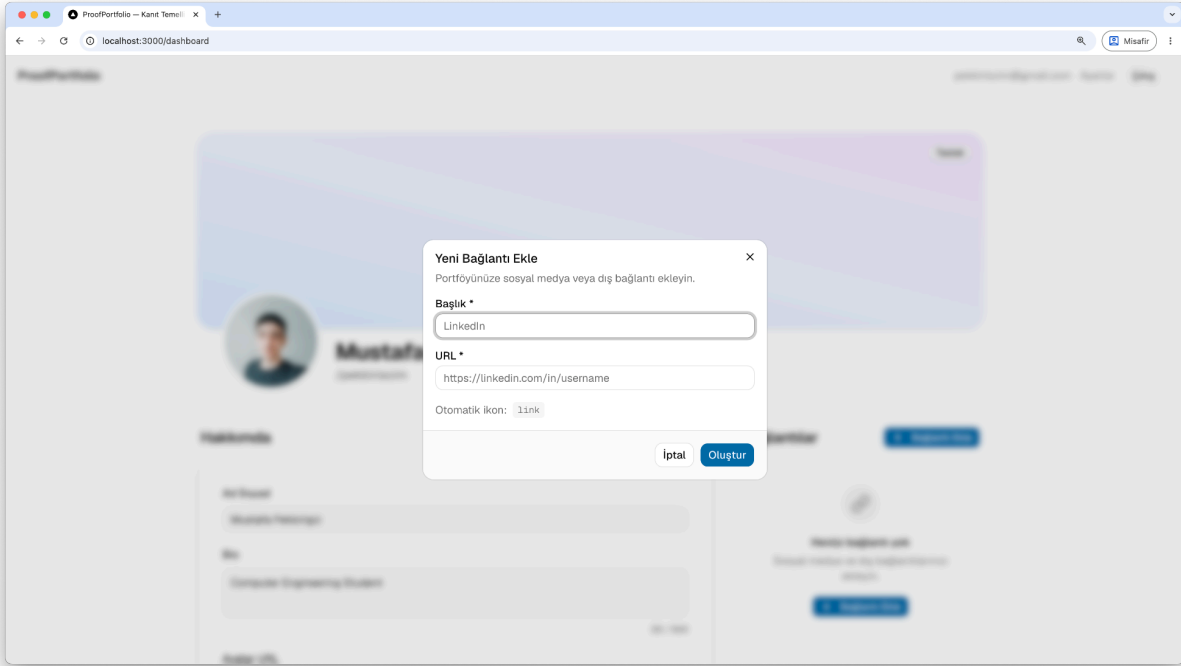
- **Başlık** (`title`) — Zorunlu, maksimum 100 karakter
- **Açıklama** (`description`) — İsteğe bağlı, maksimum 1000 karakter, textarea
- **Demo URL** (`demoUrl`) — Zorunlu `https://` prefix
- **GitHub URL** (`githubUrl`) — Zorunlu `https://` prefix
- **Görsel URL** (`imageUrl`) — İsteğe bağlı

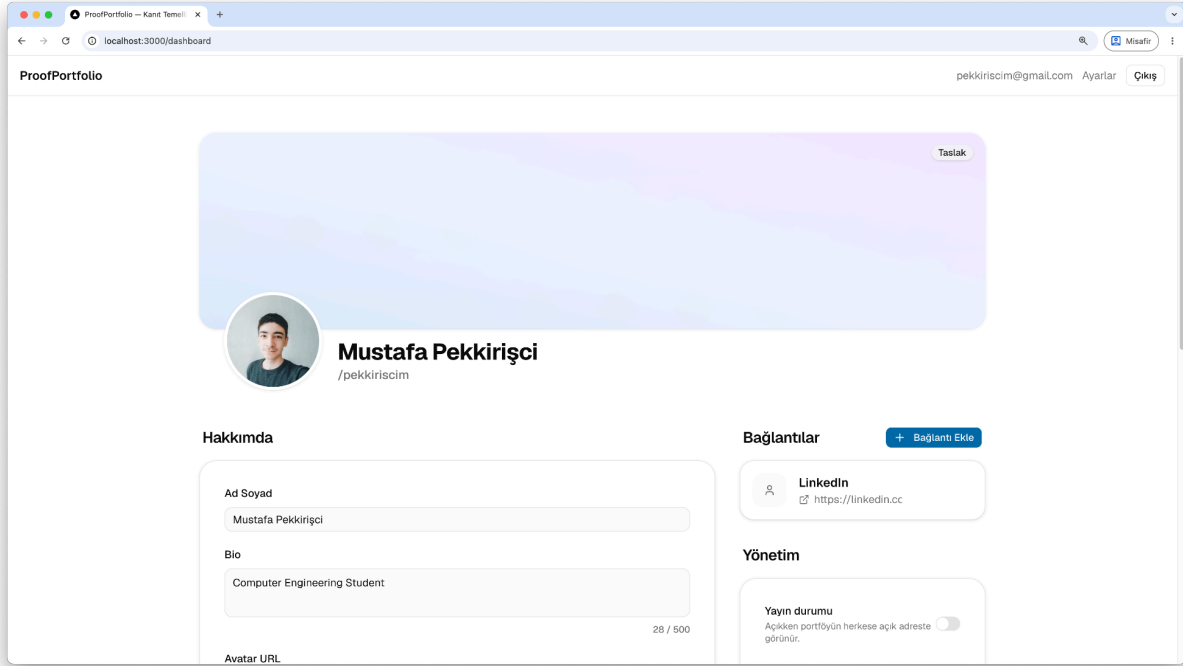
Aynı modal, `initialValues` prop'una göre "ekleme" veya "düzenleme" modunda çalışır. Modal kapandığında liste `key` prop'u sayesinde otomatik yenilenir.

**Proje Silme:** AlertDialog ile iki aşamalı onay: "Bu projeyi silmek istediğinizden emin misiniz?"

## 4.9. Dashboard — Bağlantı Yönetimi

**Aktör:** Kullanıcı (User)





### Bağlantı Listesi:

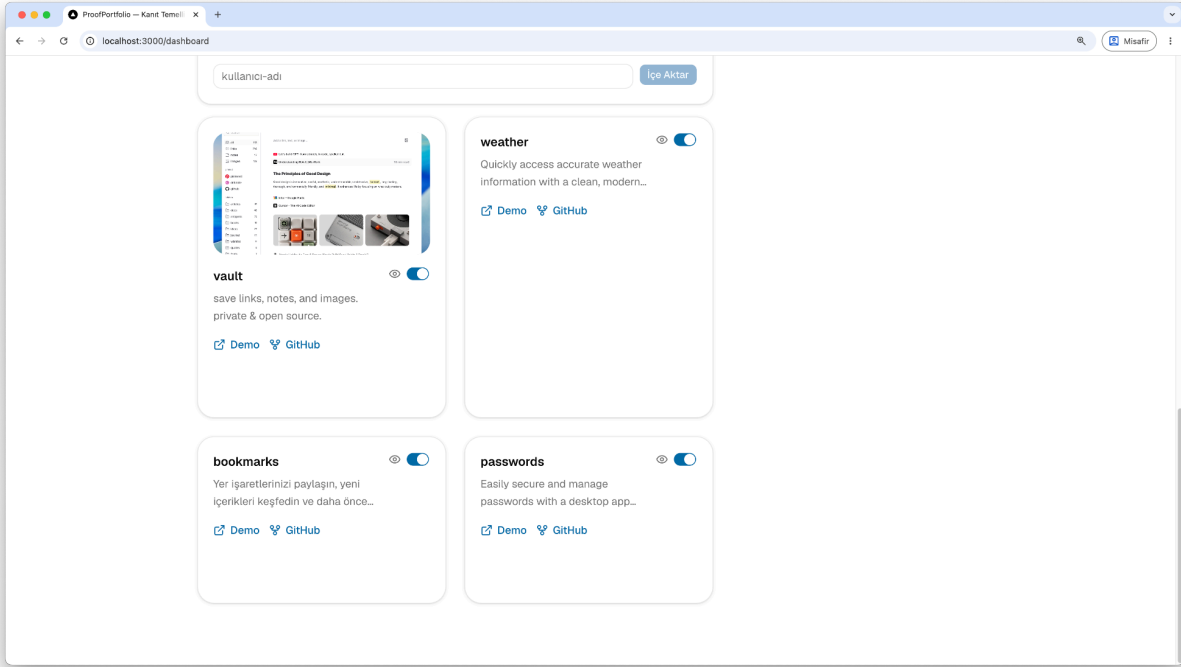
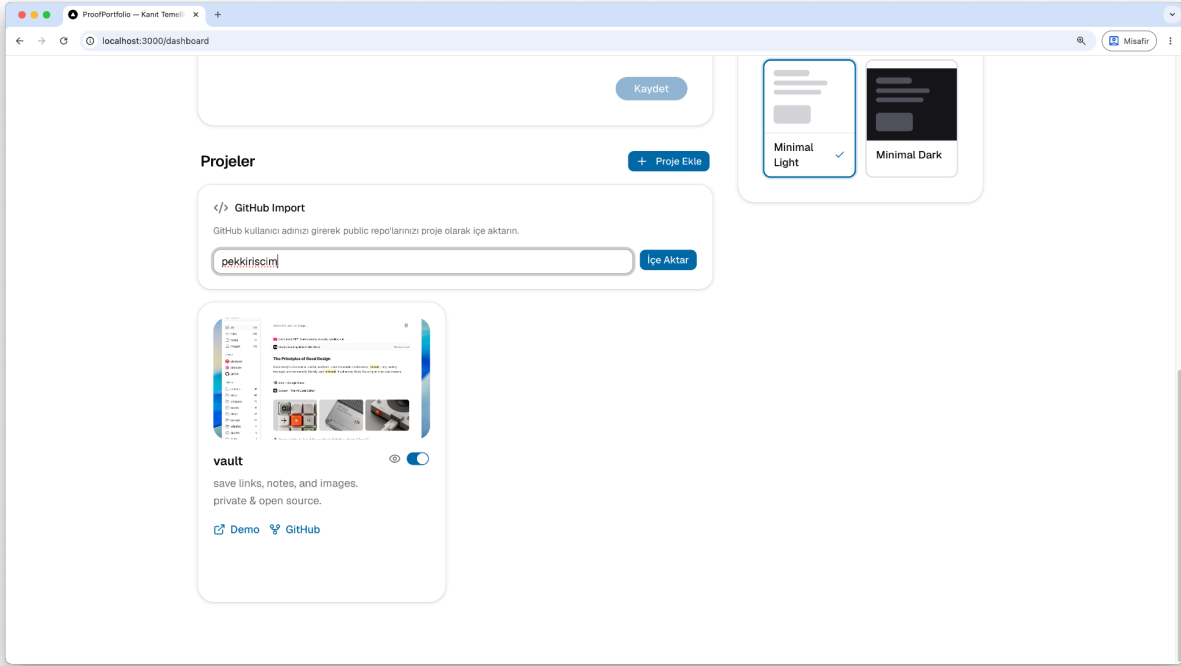
- Liste görünümü, her satırda: otomatik ikon (domain bazlı), başlık, URL, düzenle/sil butonları
- Otomatik ikon eşleştirme desteklenen platformlar: GitHub, LinkedIn, Twitter/X, Instagram, YouTube, Discord, Figma, Dribbble, Behance, Medium, Dev.to, Stack Overflow, Hashnode

### Bağlantı Ekleme/Düzenleme Modalı:

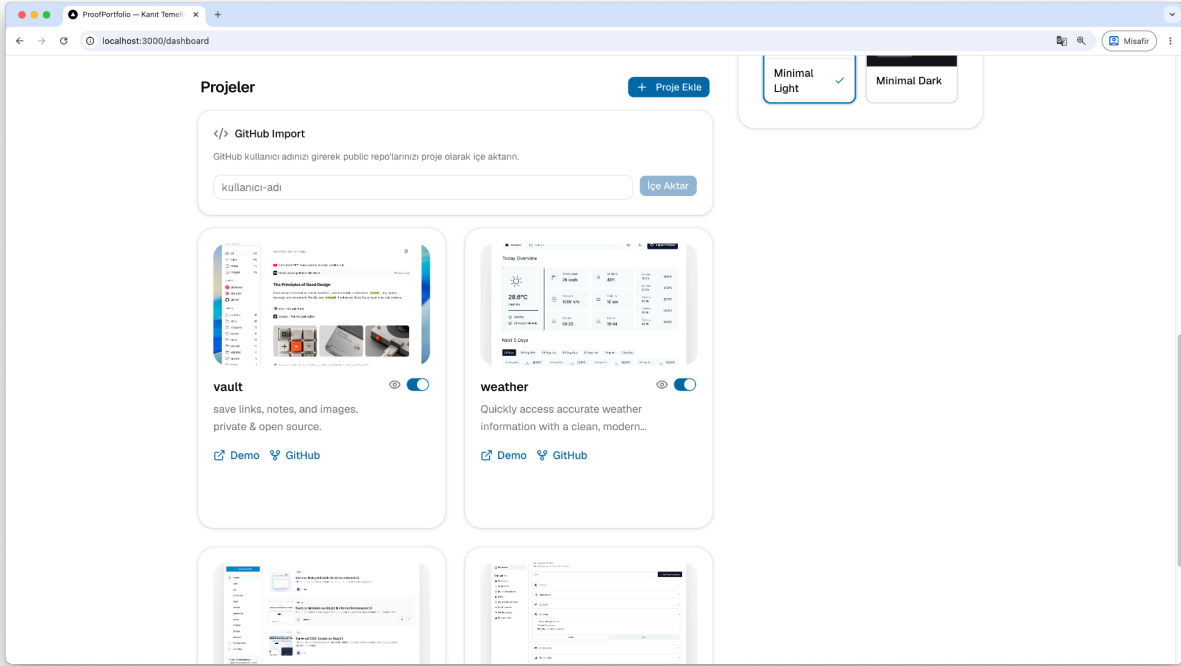
- **Başlık** (`title`) — Zorunlu, maksimum 50 karakter
- **URL** (`url`) — Zorunlu, `https://` prefix zorunlu, Zod `.url()` doğrulaması
- **İkon** (`icon`) — Boş bırakılırsa URL'den otomatik eşleştirilir

## 4.10. Dashboard — GitHub Import

**Aktör:** Kullanıcı (User)







Projeler bölümünün üstünde yer alan kompakt bir bileşen:

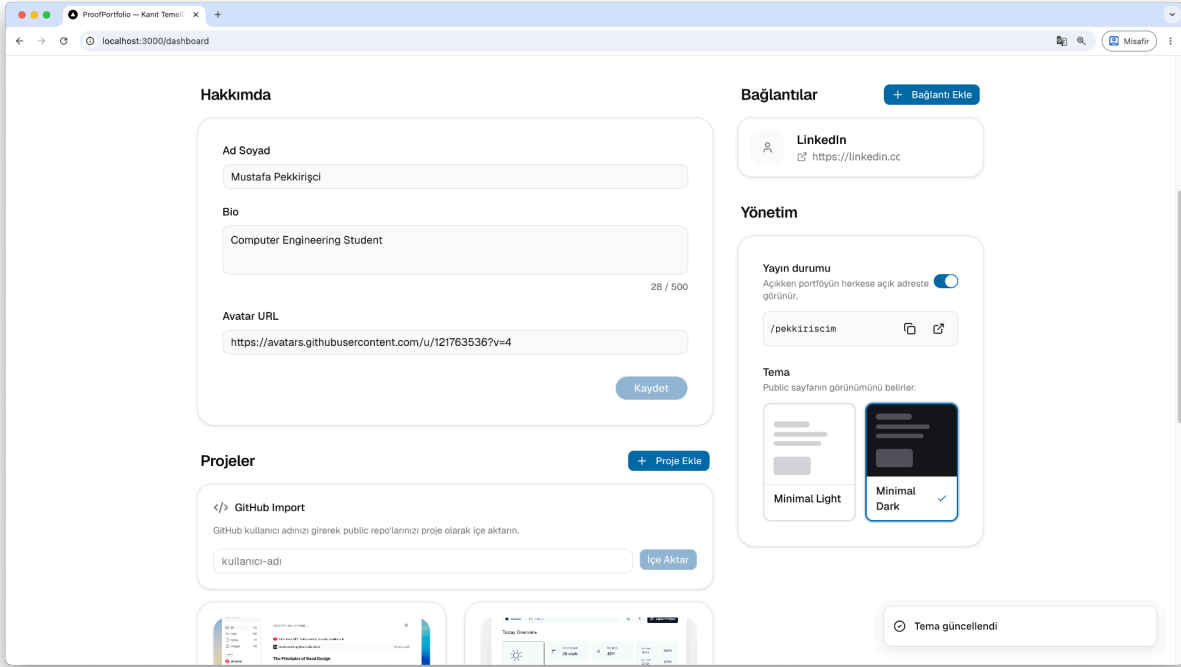
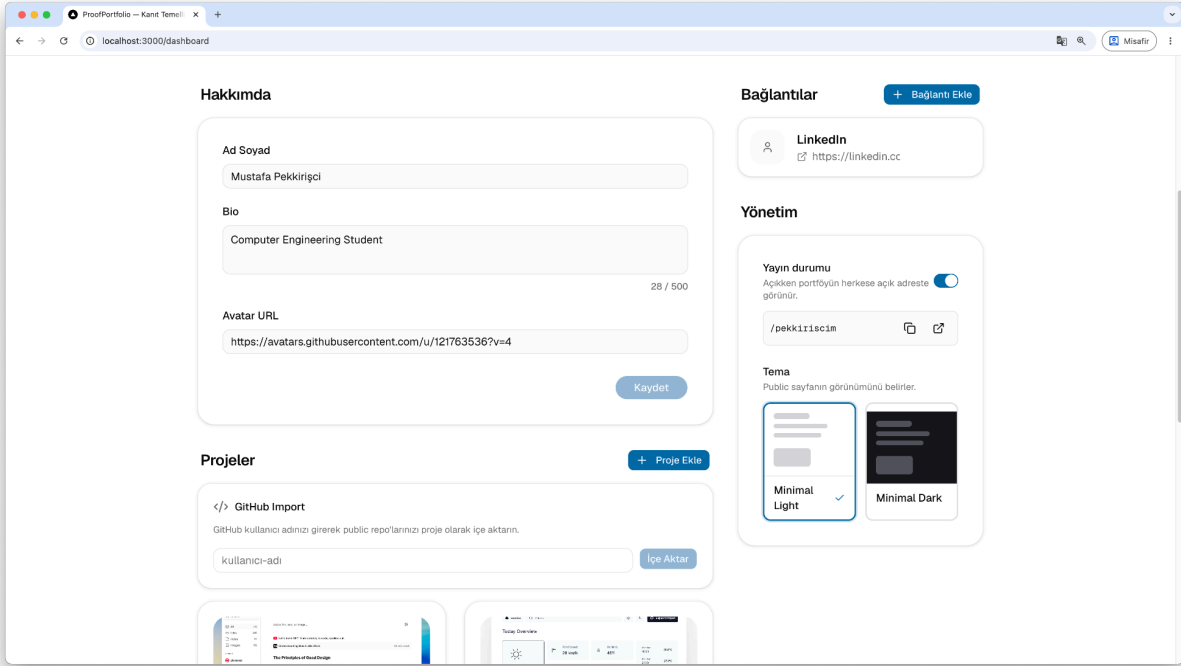
- **GitHub Kullanıcı Adı** input alanı
- **"İçe Aktar"** butonu

Arka planda:

1. `GET https://api.github.com/users/{username}/repos` çağrısı (opsiyonel `GITHUB_TOKEN` ile rate limit 5000/saat)
2. Fork'lanmış repo'lar filtrelenir
3. Her repo `{ title: repo.name, description: repo.description, githubUrl: repo.html_url }` şeklinde projeye dönüştürülür
4. Çakışma kontrolü: Aynı `githubUrl` varsa güncelle, yoksa yeni oluştur
5. Sonuç: *"X proje içe aktarıldı, Y proje güncellendi"* şeklinde toast bildirimi

## 4.11. Dashboard — Yayınlama ve Tema

**Aktör:** Kullanıcı (User)



Sağ kolondaki "Yönetim" kartı iki bileşen içerir:

### Yayınlama Toggle (PublishToggle):

- Switch bileşeni: Açık = "Yayında" / Kapalı = "Taslak"

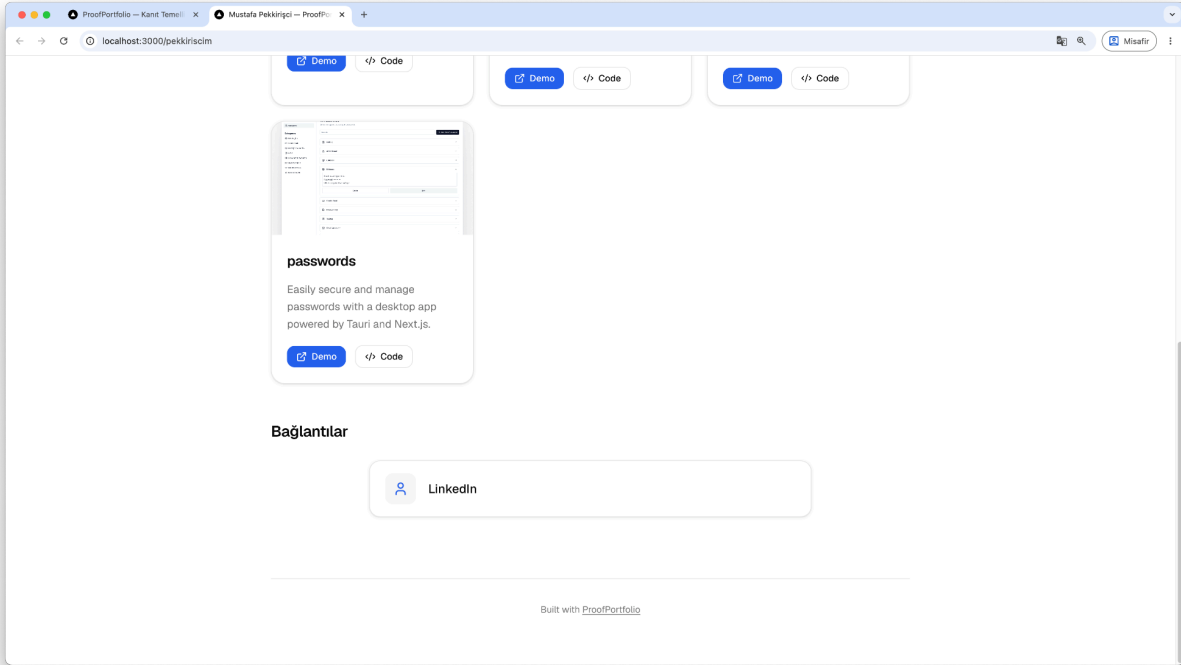
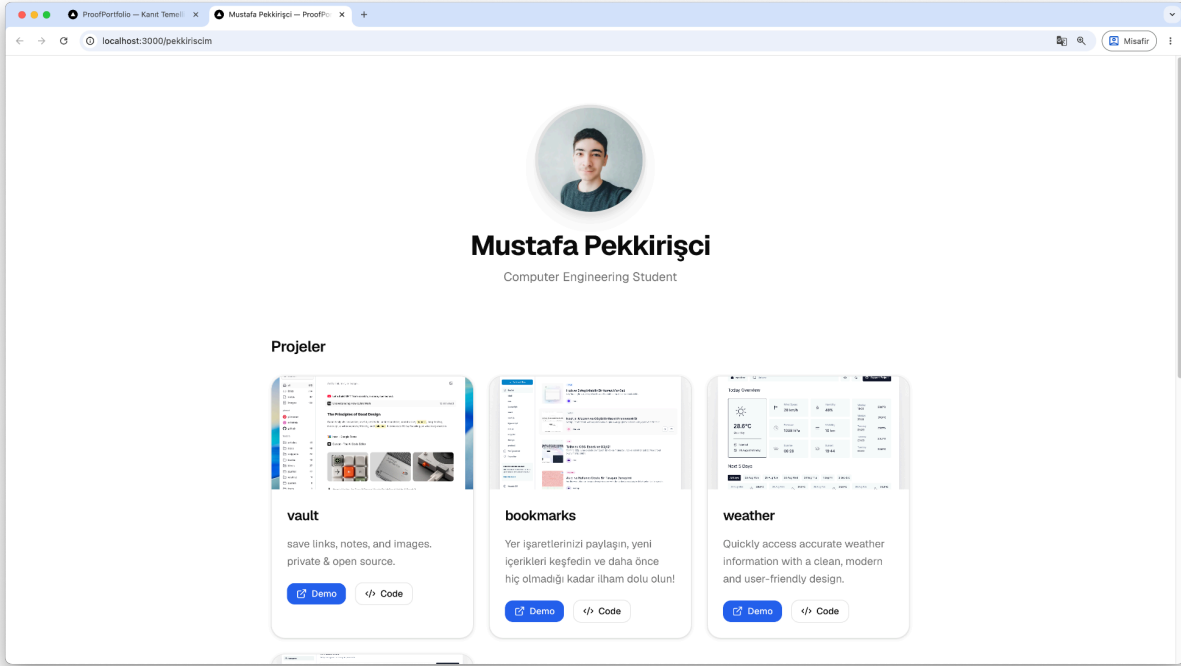
- Yayına alındığında: `revalidatePath()` ile ISR ön belleği temizlenir, public sayfa anında güncellenir
- Paylaşılabilir URL + "Kopyala" butonu gösterilir
- Hero kartındaki "Görüntüle" butonu aktif olur

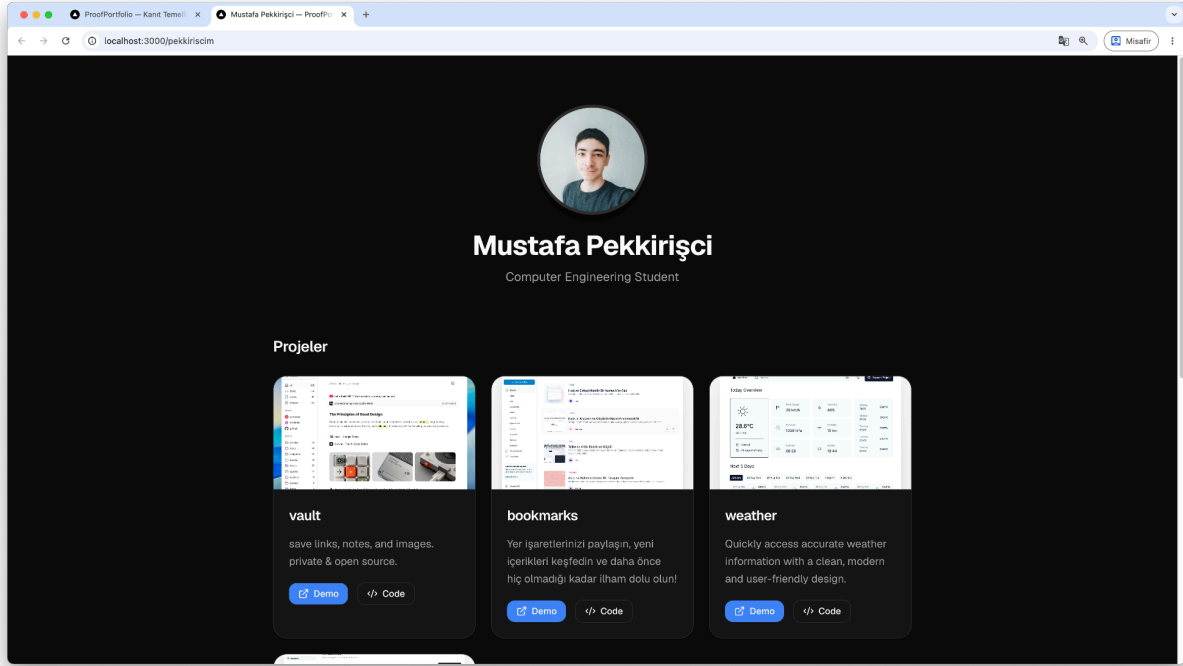
#### **Tema Seçici (ThemeSelector):**

- İki tema kartı: Minimal Light (beyaz arka plan) ve Minimal Dark (koyu arka plan)
- Seçim `PATCH /api/portfolio/[id]` ile kaydedilir
- Public portföy sayfasında `data-theme` attribute ile uygulanır

## 4.12. Public Portföy Sayfası — `/[slug]`

**Aktör:** Ziyaretçi (Guest) — Herkese açık





Portföy sahibinin yayınladığı sayfadır. **SSG/ISR** ile sunulur (60 saniye revalidate).

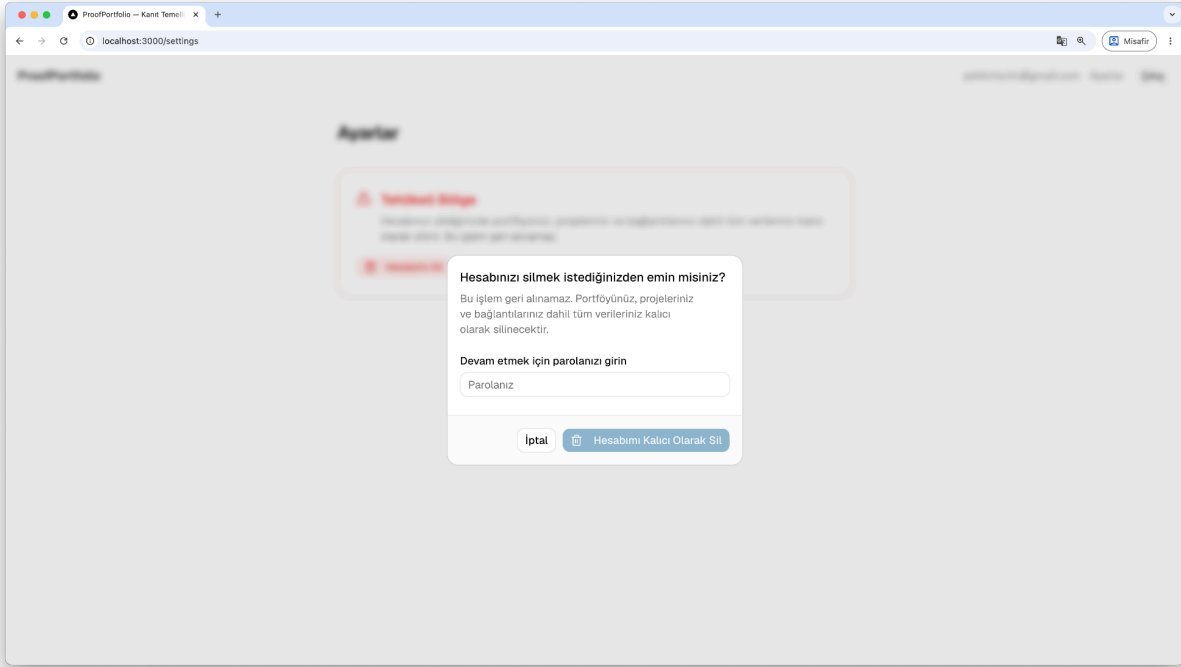
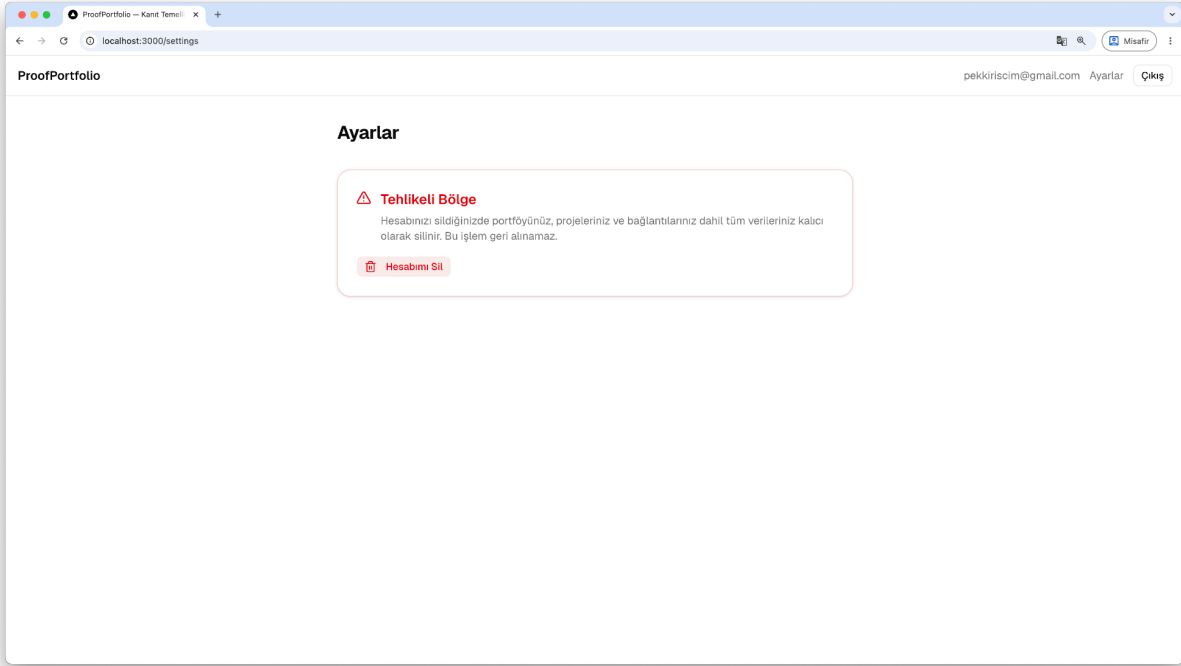
Sayfa yapısı:

- **Profil Kartı (ProfileCard):** Avatar (yuvarlak), ad soyad, biyografi metni
- **Bento Grid (BentoGrid):** Görünür projeler ve bağlantılar grid düzeninde
  - **ProjectCard:** Görsel (varsa), başlık, açıklama, demo/github link butonları
  - **LinkCard:** Platform ikonu, başlık, url (yeni sekmede açar)
- **Footer:** "Built with ProofPortfolio" bağlantısı

Tema, SSR sırasında `<html data-theme="...">` ile uygulanır; hydration mismatch önlenir. Sadece `isPublished=true` portföyler ve `isVisible=true` içerikler gösterilir. Yayında olmayan portföyler için 404 sayfası döner.

## 4.13. Ayarlar — Hesap Silme (/settings)

**Aktör:** Kullanıcı (User)



Dashboard üst menüsündeki "Ayarlar" bağlantısından erişilir.

"Tehlikeli Bölge" kartı:

- Uyarı: *"Hesabınızı sildiğinizde portföyünüz, projeleriniz ve bağlantılarınız dahil tüm verileriniz kalıcı olarak silinir. Bu işlem geri alınamaz."*
- **"Hesabımı Sil"** butonu → AlertDialog açar

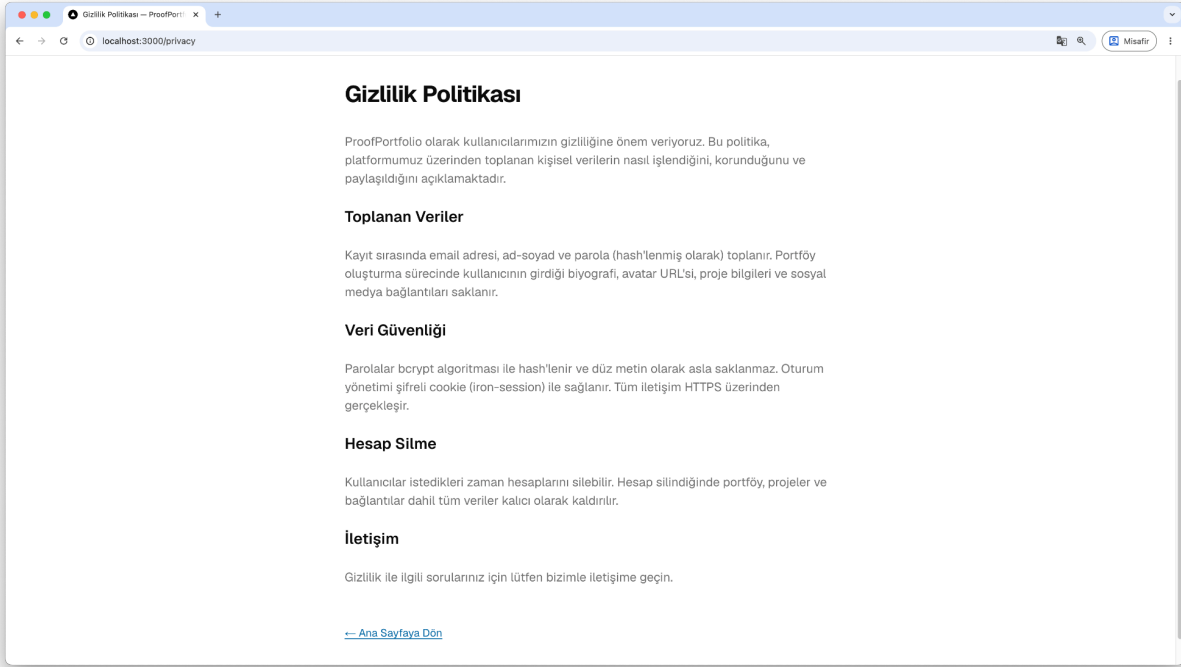
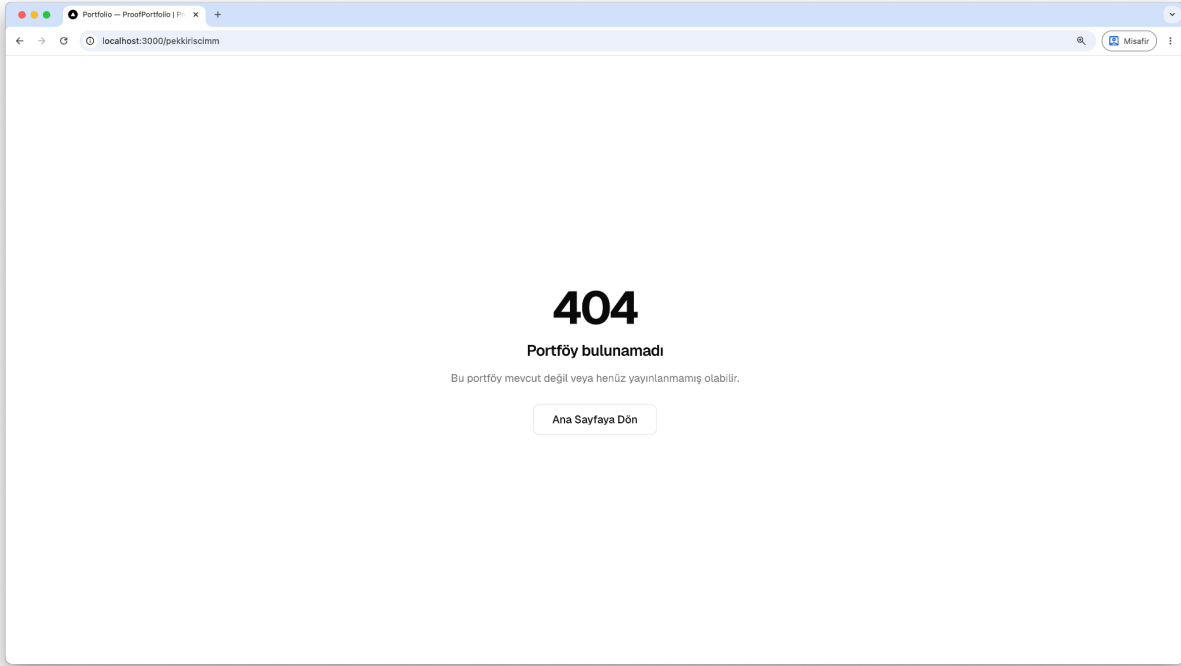
AlertDialog içinde:

- İkinci uyarı metni
- **Parola doğrulama** input'u (hesap sahibi olduğunu kanıtlamak için)
- **"İptal"** ve **"Hesabımı Kalıcı Olarak Sil"** (kırmızı) butonları

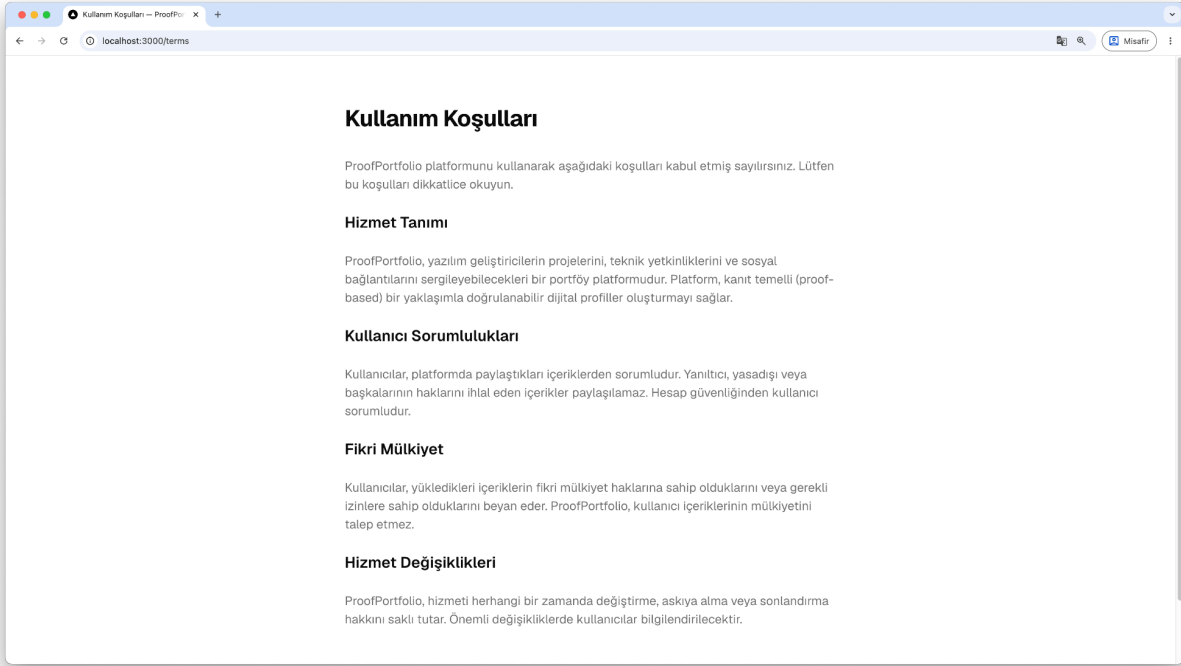
Silme işlemi **DELETE /api/account** endpoint'ini çağırır. Cascade silme ile kullanıcı, portföy, projeler ve bağlantılar veritabanından temizlenir. İşlem sonrası kullanıcı login sayfasına yönlendirilir.

#### 4.14. Hata Sayfaları ve Diğer

**Aktör:** Tüm kullanıcılar







**Global 404 Sayfası (/not-found):** Geçersiz URL'ler için minimal tasarım.

**Public Portföy 404 (/ [slug] /not-found):** "Portfolio not found or not published" mesajı.

**Global Error Boundary (/error):** Beklenmeyen sunucu hatalarını yakalar, kullanıcıya jenerik hata mesajı gösterir.

**Gizlilik Politikası (/privacy):** Platformun veri işleme politikaları.

**Kullanım Koşulları (/terms):** Hizmet kullanım şartları.

#### 4.15. Kullanıcı Roller ve Erişim Matrisi

| İşlev                   | Ziyaretçi (Guest) | Kullanıcı (User)   |
|-------------------------|-------------------|--------------------|
| Ana sayfayı görüntüleme | ✓                 | ✓                  |
| Kayıt olma              | ✓                 | ✗ (oturum açıkken) |
| Giriş yapma             | ✓                 | ✗ (oturum açıkken) |
| Parola sıfırlama        | ✓                 | ✗                  |

| İşlev                       | Ziyaretçi (Guest)       | Kullanıcı (User)   |
|-----------------------------|-------------------------|--------------------|
| Public portföy görüntüleme  | ✓ (sadece yayındakiler) | ✓                  |
| Portföy oluşturma/düzenleme | x                       | ✓ (kendi portföyü) |
| Proje CRUD                  | x                       | ✓ (kendi portföyü) |
| Bağlantı CRUD               | x                       | ✓ (kendi portföyü) |
| GitHub import               | x                       | ✓ (kendi portföyü) |
| Tema değiştirme             | x                       | ✓                  |
| Yayınlama/gizleme           | x                       | ✓                  |
| Profil güncelleme           | x                       | ✓                  |
| Hesap silme                 | x                       | ✓                  |

## 5. Kurulum Talimatları

Bu bölüm, projeyi sıfırdan kurmak isteyen bir geliştirici için adım adım talimatlar içerir.

### 5.1. Gereksinimler

| Bağımlılık | Minimum Versiyon      | Açıklama           |
|------------|-----------------------|--------------------|
| Node.js    | ≥ 18.0                | JavaScript runtime |
| npm        | ≥ 9.0                 | Paket yöneticisi   |
| PostgreSQL | 14+ (Neon serverless) | Veritabanı         |

#### Önerilen Araçlar:

- VS Code (geliştirme ortamı)
- Neon hesabı (ücretsiz: <https://neon.tech>)
- Vercel hesabı (deploy için, opsiyonel)

### 5.2. Repository'yi Klonlama

git clone <https://github.com/seйдаoksuzoglu/proof-based-portfolio-platform.git>

```
cd proof-based-portfolio-platform/src
```

### 5.3. Bağımlılıkları Yükleme

```
npm install
```

Bu komut aşağıdaki ana bağımlılıkları yükler:

| Paket                           | Amaç                              |
|---------------------------------|-----------------------------------|
| <code>next</code> 16.1.7        | Web framework (App Router)        |
| <code>react</code> 19.x         | UI kütüphanesi                    |
| <code>drizzle-orm</code> 0.45.x | ORM (PostgreSQL)                  |
| <code>iron-session</code> 8.x   | Şifrelenmiş cookie-based session  |
| <code>bcryptjs</code>           | Parola hashleme                   |
| <code>zod</code> 4.x            | Şema doğrulama                    |
| <code>react-hook-form</code>    | Form yönetimi                     |
| <code>tailwindcss</code> 4.x    | CSS framework                     |
| <code>shadcn/ui</code>          | UI bileşenleri (Radix UI tabanlı) |

### 5.4. Ortam Değişkenlerini Ayarlama

```
cp .env.example .env.local
```

Ardından `.env.local` dosyasını açın ve aşağıdaki değişkenleri doldurun:

| Değişken                  | Zorunlu                             | Açıklama                                                                                                   |
|---------------------------|-------------------------------------|------------------------------------------------------------------------------------------------------------|
| <code>DATABASE_URL</code> | <input checked="" type="checkbox"/> | Neon PostgreSQL bağlantı URL'si<br>( <code>postgresql://user:password@host/dbname?sslmode=require</code> ) |

| Değişken       | Zorunlu | Açıklama                                                                                           |
|----------------|---------|----------------------------------------------------------------------------------------------------|
| AUTH_SECRET    | ✓       | iron-session şifreleme anahtarı (min 32 karakter). Üretmek için: <code>openssl rand -hex 32</code> |
| RESEND_API_KEY | ✗       | Parola sıfırlama e-postaları için. Yoksa e-postalar konsola loglanır                               |
| EMAIL_FROM     | ✗       | Gönderen e-posta adresi                                                                            |
| GITHUB_TOKEN   | ✗       | GitHub API token. Yoksa rate limit 60 istek/saat; token ile 5000                                   |

**Not:** NEXT\_PUBLIC\_APP\_URL ortam değişkeni de gerekebilir. Geliştirme ortamında varsayılan olarak `http://localhost:3000` kullanılır.

## 5.5. Veritabanı Migration

Proje **Drizzle ORM** kullanır. Migration adımları:

# Migration dosyası oluştur (schema.ts'deki değişiklikleri algılar)

`npm run db:generate`

# Migration'ı Neon veritabanına uygula

`npm run db:migrate`

Bu işlem sonucunda aşağıdaki tablolar oluşturulur:

| Tablo     | Açıklama               | İlişkiler           |
|-----------|------------------------|---------------------|
| user      | Kullanıcı hesapları    | 1:1 → portfolio     |
| portfolio | Portföy yapılandırması | 1:N → project, link |
| project   | Proje kayıtları        | N:1 → portfolio     |

| Tablo                             | Açıklama                    | İlişkiler       |
|-----------------------------------|-----------------------------|-----------------|
| <code>link</code>                 | Bağlantı kayıtları          | N:1 → portfolio |
| <code>password_reset_token</code> | Parola sıfırlama token'ları | N:1 → user      |

Tüm foreign key'ler **ON DELETE CASCADE** ile tanımlanmıştır.

## 5.6. Seed Verisi (Opsiyonel)

Demo verisi eklemek için:

```
npm run db:seed
```

Bu komut şu verileri oluşturur:

- **1 demo kullanıcı** (bcrypt hash'li parola ile)
- **1 portföy** (yayında, light tema)
- **3 proje** (farklı görünürlük durumlarıyla)
- **4 bağlantı** (GitHub, LinkedIn, Twitter, Instagram)

**Seed verisi kullanıcı bilgileri:** `lib/db/seed.ts` dosyasında tanımlanmıştır.

## 5.7. Geliştirme Sunucusunu Başlatma

```
npm run dev
```

Sunucu varsayılan olarak <http://localhost:3000> adresinde başlar. Turbopack ile çalışır (hızlı hot-reload).

## 5.8. Diğer Yardımcı Komutlar

# TypeScript tip kontrolü

```
npm run typecheck
```

# Lint kontrolü

```
npm run lint
```

# Kod formatlama

npm run format

# Testleri alıřtırma

npm run test

# Testleri izleme modunda alıřtırma

npm run test:watch

# Drizzle Studio (veritabanı grsel arayz)

npm run db:studio

## 5.9. Testleri alıřtırma

Proje **Vitest** ile test edilmiřtir:

# Tm testleri alıřtır

npm test

# Belirli bir test dosyası

npx vitest run tests/unit/auth-controller.test.ts

### Test Kapsamı:

| Test Tr         | Dosya Sayısı | Kapsam                                                              |
|-------------------|--------------|---------------------------------------------------------------------|
| Birim Test        | 6            | Controller sınıfları (auth, portfolio, project, link, icon mapping) |
| Entegrasyon Testi | 4            | API route handler'lar (auth, portfolio, projects, links)            |

Toplam **10+ test dosyası** ile controller'ların her public metodu ve tm API endpoint'leri test edilmiřtir.

## 5.10. Production Build

# Production build oluştur

```
npm run build
```

# Production sunucusunu başlat

```
npm run start
```

Build çıktısı `.next/` klasöründe oluşturulur.

## 5.11. Vercel Deploy

Platform Vercel'e deploy edilmek üzere yapılandırılmıştır:

1. Vercel hesabınıza giriş yapın (<https://vercel.com>)
2. "Add New Project" → GitHub repository'sini bağlayın
3. Framework: **Next.js** (otomatik algılanır)
4. Root Directory: `src/` (proje kök dizini)
5. Environment Variables:
  - `DATABASE_URL` → Neon bağlantı URL'si
  - `AUTH_SECRET` → 32 karakterli rastgele anahtar
  - `NEXT_PUBLIC_APP_URL` → Vercel domain'i
6. Deploy → Her `main` branch push'unda otomatik deploy
7. Preview Deploy: Her PR için otomatik preview URL oluşturulur

## 5.12. Proje Klasör Yapısı

`src/`

|                |                                  |
|----------------|----------------------------------|
| — app/         | # Next.js App Router sayfaları   |
| — (auth)/      | # Giriş, kayıt, parola sıfırlama |
| — (dashboard)/ | # Dashboard, ayarlar             |
| — [slug]/      | # Public portföy (SSG/ISR)       |
| — api/         | # Tüm API route handler'lar      |
| — privacy/     | # Gizlilik politikası            |

|                     |                                           |
|---------------------|-------------------------------------------|
| — terms/            | # Kullanım koşulları                      |
| — error.tsx         | # Global error boundary                   |
| — not-found.tsx     | # 404 sayfası                             |
| — layout.tsx        | # Root layout                             |
| — components/       | # React bileşenleri                       |
| — ui/               | # shadcn/ui (~60 bileşen)                 |
| — auth/             | # LoginForm, RegisterForm                 |
| — dashboard/        | # Proje/link yönetimi, profil, onboarding |
| — portfolio/        | # Public view bileşenleri                 |
| — lib/              | # İş mantığı ve altyapı                   |
| — controllers/      | # Auth, Portfolio, Project, Link, User    |
| — db/               | # Şema, singleton, seed                   |
| — validators/       | # Zod şemaları                            |
| — auth.ts           | # iron-session helpers                    |
| — errors.ts         | # Custom hata sınıfları                   |
| — github.ts         | # GitHub API client                       |
| — tests/            | # Test dosyaları                          |
| — unit/             | # Controller birim testleri               |
| — integration/      | # API route entegrasyon testleri          |
| — drizzle/          | # Migration dosyaları                     |
| — drizzle.config.ts | # Drizzle konfigürasyonu                  |
| — middleware.ts     | # Route protection                        |



└─ package.json

# Bağımlılıklar ve script'ler

## 6. Görev Dağılımı

### MUSTAFA PEKKİRİŞCİ

#### Analiz Raporu:

- Kullanım Durumu ve Sınıf Diyagramlarının çizimine katıldı.
- Durum ve Aktivite Diyagramlarının hazırlanmasında görev aldı.
- Sıra Diyagramının çıkarılmasına katkı sağladı.

#### Tasarım Raporu:

- Kalıcı Veri Yönetimi çizimini yaptı.
- Sınıf Arayüzleri altındaki kontrol sınıflarının tasarım detaylarını hazırladı.

#### İmplementasyon:

- Sistemin genel mimari kararlarını verdi ve altyapıyı kurdu.
- Veritabanı şemasını tasarladı ve entegre etti.
- Kimlik doğrulama altyapısını geliştirdi.

#### Final Raporu:

- Kullanıcı Kılavuzu ve Kurulum Talimatları dokümantasyonunu yazdı.

### BUSE YANAL

#### Analiz Raporu:

- Metinsel Gereksinimlerin Yazımı aşamasında görev aldı.
- Kullanım Durumu ve Sınıf Diyagramlarının oluşturulmasına katıldı.

#### Tasarım Raporu:

- Giriş, Tasarım Desenleri, Sözlük ve Referanslar kısımlarının yazımını yaptı.
- Raporun genel koordinasyonunu ve son birleştirme işlemlerini üstlendi.

#### İmplementasyon:

- Portföy, Proje ve Bağlantı modüllerinin veri işlemlerini kodladı.
- Adres uzantısı kontrol mekanizmasını yazdı.
- GitHub üzerinden proje verisi çekme özelliğini geliştirdi.

#### Final Raporu:

- Giriş bölümünü yazdı.

## **ŞEYDA ÖKSÜZOĞLU**

### **Analiz Raporu:**

- Kullanım Durumu, Sınıf, Durum ve Aktivite Diyagramlarının hazırlanmasında yer aldı.

### **Tasarım Raporu:**

- Küresel Kontrol Akışı diyagram ve metinlerini hazırladı.
- Son Nesne Tasarımı çalışmasını yaptı.
- Paketler ve Katmanlar içeriklerini yazdı.

### **İmplementasyon:**

- Proje yönetiminde iş takibini yaptı.
- Geliştirme sürecinin dokümantasyonunu tuttu ve risk yönetimini üstlendi.
- Arayüzdeki küçük kullanıcı arabirimi parçalarının kodlamasını yaptı.

### **Final Raporu:**

- Yaptıklarınız, Yapamadıklarınız ve Nedenleri kısmını yazdı.

## **ELİF İPEK KAYNAR**

### **Analiz Raporu:**

- Metinsel Gereksinimlerin Yazımı kısmında çalıştı.
- Kullanıcı Arayüzü taslak çizimlerini üstlendi.

### **Tasarım Raporu:**

- Erişim Kontrolü ve Güvenlik mimarisini kurguladı.
- Sınıf Arayüzleri altındaki kontrol sınıflarının detaylarını tasarladı.

### **İmplementasyon:**

- Kimlik doğrulama servislerini geliştirdi.
- Şifre güvenliği için kriptografik algoritma entegrasyonunu yaptı.
- Oturum yönetimi altyapısını kodladı.

### **Final Raporu:**

- Yapay Zeka Kullanımı kısmını hazırladı.

## **RABİA ŞAHİN**

### **Analiz Raporu:**

- Metinsel Gereksinimlerin Yazımı aşamasına katkı sağladı.

### **Tasarım Raporu:**

- Donanım Yazılım Eşlemesi şemasını çizdi.
- Sınıf Arayüzleri altındaki sınır sınıflarının detaylarını hazırladı.

#### **İmplementasyon:**

- Dinamik rota yapısını kodladı.
- Uygulamanın tema sistemini kurdu.
- Ana sayfa düzeni için modüler grid arayüz bileşenini geliştirdi.

#### **Final Raporu:**

- Proje Deneyimi bölümünü yazdı.

### **ALİ AKBAYRAK**

#### **Analiz Raporu:**

- Sıra Diyagramının hazırlanmasına katkı sağladı.

#### **Tasarım Raporu:**

- Alt Sistem Ayırıştırması kısmını hazırladı.
- Nesne Tasarım Kararları analizlerini yazdı.

#### **İmplementasyon:**

- Kullanıcı Giriş ve Kayıt Olma modüllerini geliştirdi.
- Kullanıcı Paneli arayüzünü kodladı.

#### **Final Raporu:**

- Görev Dağılımı kısmının yazımını üstlendi.

## **7. Yaptıklarımız, Yapmadıklarımız ve Nedenleri**

### **7.1. Gerçekleştirilen ve Gerçekleştirilemeyen Gereksinimler**

Tasarım raporunda tanımladığımız temel gereksinimlerin büyük çoğunluğu MVP kapsamında hayata geçirildi:

- E-posta/parola ile kayıt, giriş ve iron-session tabanlı oturum yönetimi
- Parola sıfırlama akışı (Resend ile e-posta gönderimi)
- Kullanıcı başına tek portföy oluşturma; slug benzersizlik ve rezerve kelime kontrolü
- Profil bilgilerini (ad, bio, avatar) düzenleme
- Proje ekleme/düzenleme/silme ve görünürlük aç-kapa (CRUD)
- Link ekleme/düzenleme/silme (CRUD)
- Portföy yayınlama/gizleme
- Tema seçimi (sınırlı sayıda temayla)
- Public portföy görüntüleme

Buna karşılık, tasarım aşamasında planladığımız bazı özellikleri MVP süresi içinde tamamlamadık:

- **Tema seçenekleri sınırlı kaldı.** Portföyler için çeşitli tema alternatifleri sunmayı planlamıştık; tema sisteminin altyapısı (Tailwind v4 `@theme` ve `ThemeProvider`) hazır olmasına rağmen, ek temaları tasarlayıp test edecek zamanı bulamadığımız için yalnızca sınırlı sayıda temayı tamamlayabildik.
- **GitHub'dan proje import etme özelliği kısmen gerçekleştirildi.** Tasarımda (`importFromGitHub`, Bölüm 3.4.1) kullanıcının public repolarını GitHub API üzerinden çekip otomatik olarak proje nesnelere dönüştürmeyi öngörmüştük. Bu özelliği ekledik; kullanıcı GitHub kullanıcı adını girdiğinde repolar portföye aktarılıyor ve repo bağlantısı (`githubUrl`) oluşturuluyor. Ancak repo açıklamalarının (`description`) otomatik olarak portföye yazılması kısmını tam olarak çalışır hale getiremedik; bu alan şu an boş geliyor ve kullanıcının açıklamayı elle girmesi gerekiyor. Temel akış çalışsa da, planladığımız "tek tıkla tam dolu proje kartı" hedefine ulaşamadık.
- **GitHub OAuth ile giriş eklenemedi.** Projenin başında kimlik doğrulamayı GitHub OAuth ile yapmayı planlamıştık; fakat OAuth akışını (callback yönetimi, token saklama, provider konfigürasyonu) kurmak beklediğimizden karmaşık çıktı ve MVP'de sosyal girişe gerçek bir ihtiyacımız yoktu. Bu nedenle e-posta/parola tabanlı iron-session çözümünde kaldık. (Bu kararın teknoloji tarafı Bölüm 7.3'te ayrıca ele alınmıştır.)

## 7.2. Tasarımda Değişen Sınıflar / Diyagramlar

- Sınıf diyagramına yeni bir Control sınıfı eklendi: `UserController`. Tasarım raporunda (Bölüm 3.4.1) Control katmanı yalnızca `AuthController`, `PortfolioController`, `ProjectController`, `LinkController` ve `SessionHelpers`'tan oluşuyordu. Profil güncellemesini (`fullName`, `bio`, `avatarUrl`) başta `PortfolioController.updatePortfolio()` üzerinden ele almayı düşünmüştük; fakat bunun bir portföy değil kullanıcı işlemi olduğunu ve route handler'ın doğrudan `db`'ye dokunmaması (BCE kuralı) için ayrı bir giriş noktasına ihtiyaç olduğunu fark ettik. Bu nedenle `UserController` sınıfını (`updateProfile()`, `getProfile()`) ekledik; `getProfile()` ayrıca `passwordHash` alanını dışarıya sızdırmayacak şekilde tasarlandı.
- **`PortfolioController`'a `getByUserId()` metodu eklendi.** Tasarımdaki `PortfolioController`, portföyü yalnızca slug üzerinden (`getPublicPortfolio`) getiriyordu. Ancak Dashboard sayfasının, oturum açan kullanıcının **kendi** portföyünü (yayında olsun olmasın) yükleyebilmesi gerekiyordu. Bu durum için `getByUserId(userId): Promise<Portfolio | null>` metodunu ekledik. Portföyü olmayan yeni kullanıcı için `NotFoundError` fırlatmak yerine `null` döndürmesi, onboarding akışında "henüz portföy yok" durumunu doğal şekilde ele almamızı sağladı.
- **ER diyagramına beşinci bir tablo eklendi: `passwordResetToken`.** Tasarım raporundaki ER diyagramı (Bölüm 2.3) dört tablodan oluşuyordu: `users`, `portfolios`, `projects`, `links`. Parola sıfırlama akışını (`AuthController`'daki `requestPasswordReset/confirmPasswordReset`) güvenli biçimde uygulamak için

token'ları hash'lenmiş halde saklayan, tek kullanımlık ve süreli (`id`, `userId`, `tokenHash`, `expiresAt`, `usedAt`) yeni bir tabloya ihtiyaç duyduk. Bu tablo tasarım aşamasındaki ER diyagramında yer almıyordu.

- **Boundary (UI) bileşenleri tasarımdaki listenin ötesine genişledi.** Son Nesne Tasarımı'ndaki (Bölüm 3.2) Dashboard bileşenleri `LoginForm`, `RegisterForm`, `ProjectModal`, `LinkModal`, `ProfileEditor` ve `ThemeSelector` ile sınırlıydı. Gerçek geliştirmede bunlara ek olarak `OnboardingModal` (ilk girişte slug seçimi + tema), `PublishToggle`, `LogoutButton`, `ProjectList`, `LinkList` ve `EmptyState` bileşenleri ortaya çıktı. Bu bileşenler işlevsel olarak gerekiyordu ancak tasarım aşamasındaki yüksek soyutlama düzeyinde görünmüyordu.
- **Middleware'in davranışı kontrol akışı / sıralama diyagramından farklılaştı.** Tasarım raporundaki sıralama diyagramı (Bölüm 2.5), oturum doğrulamasının (`iron-session` ile `oturum doğrula`) doğrudan `middleware.ts` içinde yapıldığını gösteriyordu. Implementasyonda ise Edge runtime'da `iron-session` çözümlemesinin (`decrypt`) maliyetli olduğunu gördük; bu yüzden middleware yalnızca cookie'nin **varlığını** kontrol edecek şekilde sadeleştirildi, asıl yetki doğrulaması Controller seviyesinde `requireAuth()` ile yapıldı. Ayrıca public rotalar için bir `PUBLIC_API_PATHS` beyaz listesi ve performans amacıyla `config.matcher` ile path daraltması eklendi.
- **`ProjectController.importFromGitHub()` metodu kısmen uygulandı.** Tasarımdaki metod repo verilerini `title=repo.name`, `description=repo.description`, `githubUrl=repo.html_url` eşlemesiyle eksiksiz çekmeyi öngörüyordu. Implementasyonda repo bağlantısı ve temel kayıt oluşturma çalıştı; ancak `description` alanının otomatik doldurulması beklediğimiz gibi sonuç vermedi (GitHub API'den dönen veriyi kendi `Project` modelimize eşleştirirken bu alan boş kalıyor). Metodun imzası ve genel akışı tasarımla uyumlu kaldı, yalnızca veri eşleştirmesinin bir bölümü eksik tamamlandı.

### 7.3. Vazgeçtiğimiz / Değiştirdiğimiz Teknolojiler ve Nedenleri

- **Python backend yerine full-stack Next.js.** Başta backend'i Python (Django/FastAPI), frontend'i ayrı bir React uygulaması olarak planlamıştık. Ancak iki ayrı deploy, CORS yapılandırması ve Python–TypeScript arasında sürekli tip senkronizasyonu MVP için gereksiz yük getiriyordu. Bunun yerine her şeyi Next.js 16 (App Router) ile tek projede topladık; aynı Zod şemalarını istemci ve sunucuda yeniden kullanarak uçtan uca tip güvenliği sağladık. Trade-off: Python'ın güçlü veri işleme/ML ekosistemine erişimimiz kalmadı (MVP'de ihtiyaç yoktu).
- **Prisma yerine Drizzle ORM.** Prisma daha yaygın ve öğrenmesi kolay olsa da, Neon (serverless PostgreSQL) ile verimli çalışması çoğu zaman Prisma Accelerate gibi ek servis gerektiriyor ve büyük bundle boyutu cold-start sürelerini uzatıyordu. Drizzle, Neon'un serverless driver'ıyla doğrudan çalışıp küçük bundle üretiyor. Trade-off: Drizzle'ın API'si daha az sezgisel; bazı sorgularda doğrudan SQL bilgisi gerekti.

- **NextAuth yerine iron-session.** Projenin başında GitHub OAuth ile giriş planlamış ve NextAuth'u değerlendirmiştik; ancak OAuth akışını (callback, token saklama, provider konfigürasyonu) kurmak beklediğimizden karmaşık çıktı ve MVP'de sosyal girişe ihtiyacımız yoktu. Bu yüzden çok daha sade olan iron-session'a geçtik. *Trade-off:* İleride OAuth eklenmek istenirse auth katmanı kısmen yeniden yazılmalı.
- **CSS Modules yerine Tailwind CSS v4.** shadcn/ui zaten Tailwind üzerine kurulu olduğundan CSS Modules seçmek bileşenleri sıfırdan uyarlamayı gerektirirdi; ayrıca Tailwind v4'ün **@theme** sistemi tema yönetimini kolaylaştırdı. *Trade-off:* v4 henüz yeni olduğu için dokümantasyonu az olgunlaşmıştı.

#### 7.4. Kullanmadığımız / Ertelelediğimiz External API ve Kütüphaneler

- **Mesaj kuyruğu / arka plan işçileri / WebSocket (Redis, worker süreçleri):** Tasarım raporunda da belirttiğimiz gibi (Bölüm 2.5), asenkron mimariyi bilinçli olarak MVP dışında bıraktık. Tüm iş akışları senkron request-response modeliyle yeterince hızlı tamamlandığından, ek altyapı maliyetine değmeyeceğine karar verdik.
- **OAuth sağlayıcıları (GitHub/Google ile giriş):** iron-session kararının doğal sonucu olarak MVP'de eklenmedi; ileri bir sürüme bırakıldı.
- **Kullandığımız dış servisler:** Resend (parola sıfırlama e-postaları), Neon (managed PostgreSQL) ve GitHub REST API (proje import; temel akış çalışıyor, açıklama eşleştirmesi eksik).

#### GitHub Repository

<https://github.com/seйдаoksuzoglu/proof-based-portfolio-platform>